Document d’architecture technique

Table of Contents

Q1: En partant des pré-requis, que proposez-vous comme séparation en quartiers fonctionnels de votre application et en termes de micro-services? . . . 2
- Vocabulaire métier et user stories . . . 2
- Découpage en quartier fonctionnels . . . 8
- Justification de l’architecture . . . 9

Q2: Comment gérer le système d’authentification avec un OIDC? . . . 10
Q3: Communication Inter micro-services . . . 11
Q4: Comment tracer les logs et les requêtes? . . . 13
Q5: Comment ajouter un moteur de recherche? . . . 15
- Moteur de recherche . . . 15
- Intégration dans le frontend . . . 17

Q6: Comment mettre en place l’architecture de runner . . . 20
- Définition du runner . . . 20
- Architecture . . . 20
- Enregistrement des runners . . . 21

Q7: Comment gérer les données? . . . 22
- Organisation des bases de données . . . 22
- SGBD utilisés . . . 23

Q8: Comment ajouter une application mobile dans le système . . . 28
- Fonctionalités et mockup visuels . . . 28
- Flows de fonctionnement . . . 32
- API . . . 33
- Authentification . . . 33

Q9: Comment gérer les problématiques de sécurité . . . 34
- Sécurisation du réseau . . . 34
- Gestion des secrets . . . 36
- Protection contre les attaques DDoS . . . 37

Q10: Comment intégrer les applications UI? . . . 39
Bibliographie/Webographie/Liens . . . 40

Q1: En partant des pré-requis, que proposez-vous comme séparation en quartiers fonctionnels de votre application et en termes de micro-services?

Vocabulaire métier et user stories

Suite à de nombreuses incompréhensions, nous avons redéfini une grande partie du vocabulaire métier, nous en avons aussi profité pour redéfinir nos user stories.

Voici les définitions du vocabulaire métier que vous serez succesptible de trouver dans nos user stories et dans le reste du document.

- **Invité** : Client n'ayant pas de compte sur la plateforme
- **Utilisateur** : Client ayant un compte sur la plateforme
- **Administrateur** : Utililisateur privilégié ayant des droits CRUD sur toutes les ressources de la plateforme
- **Équipe** : Regroupement d'utilisateurs composé d'un capitaine et de membres
- **Évaluation** : Une évaluation destinée à des utilisateurs
- **Test** : Il s'agit d'un groupe ordonné de contenus, noté, conçu pour évaluer les utilisateurs. Il est créé par un créateur d'évaluation.
- **Candidat** : Utilisateur participant à une évaluation ou a un test
- **Campagne de tests** : exposition d'un groupe utilisateurs à un test
- **Composant** : Élémént affiché à l'utilisateur, il peut s'agir de markdown, d'un éditeur de code etc…
- **Contenu** : Ensemble cohérent de composants
- **Module** : Un groupement de contenus, peut également contenir d'autres modules appelés sous-modules
- **PolyPoints** : Points gagnés par les utilisateurs lorsqu'ils réussissent des défis de composant
- **Validateur** : Élément caché de l'utilisateur, associé à un composant, il permet de vérifier la validité des réponses de l'utilisateur et récompense les bonnes réponses avec des PolyPoints.

En tant que…	Je veux…	Afin de…
Invité	Créer un compte	D'utiliser l'application
Invité	Créer un compte avec un compte Google/Polytech	D'utiliser l'application sans avoir à créer un mot de passe et sans avoir à saisir mes informations personnelles

En tant que…	Je veux…	Afin de…
Utilisateur	Mettre à jour mes adresses mails	Pour récupérer mon compte en cas de perte de mot de passe ou en cas de problèmes d'email
Utilisateur	Recevoir un mail lors de bienvenue après inscription	Être averti que mon inscription à bien été faite
Utilisateur	Changer mon nom, mon mot de passe, mon language de programmation et ma bio	Mes informations personelles soient à jourwzJB7BBo4W9uUjm+rBMS3H4ZoDTE2dM2A+JbfE515doK
Utilisateur	Delete my account	Ne plus avoir de trace sur l'application
Utilisateur	Me connecter grâce une adresse mail et un mot de passe/grâce à un compte Google ou Polytech	Je puisse accèder à toutes les fonctionnalités de l'application
Utilisateur	Me déconnecter de mon compte	Éviter les accès non autorisés depuis le navigateur utilsé.
Utilisateur	Récupérer mon mot de passe via un mail	Je puisse récupérer mon compte en cas de perte de mon mot de passe
Utilisateur	Créer une nouvelle équipe	Réunir un groupe d'utilisateurs et parciper au leaderboard des équipes (somme des points de chaque utilisateur).
Capitaine	Invite d'autres utilisateurs à être membre de mon équipe	Mon équipe s'agrandisse
Capitaine	Exclure un membre de mon équipe	Enlever un membre problématique / non actif / qui n'est plus en adéquation avec le groupe.
Capitaine	Donner le rôle de capitaine à un autre membre de mon équipe	Je sois disposé de ces fonctions
Capitaine	Supprimer mon équipe	Ne plus avoir de traces de cette équipe, pour quelconque raison.
Capitaine	Changer le nom et la description de mon équipe	Les informations de l'équipe reste à jour
Utilisateur	Accepter ou décliner une invitation à une équipe	Ajoute les PolyPoints (précédents) et gagnés de l'utilisateur à l'équipe
Utilisateur	Pouvoir quitter une équipe	Je ne sois plus associé à un groupe d'utilisateur
Utilisateur	Voir les points de mon équipe	Constater l'avancement des membres de l'équipe
Utilisateur	Voir le classement des équipes	Je vois le placement de mon équipe vis-à-vis des autres

En tant que…	Je veux…	Afin de…
Utilisateur	Voir le classement interne des membres de l'équipe	Voir qui a participé le plus dans l'équipe, concurrence interne
Utilisateur	Voir la liste des exercices disponibles	Je puisse choisir un exercice à faire
Utilisateur	Voir la liste des modules disponibles	Je puisse choisir un module à faire
Utilisateur	Voir les sous-modules et les exercices d'un module	Trouver les étapes à faire pour compléter le module
Utilisateur	Voir la liste des évaluations disponibles	Je puisse choisir une évaluation à passer
Utilisateur	Voir les derniers exercices / modules mis en ligne	Voir le nouveau contenu
Utilisateur	Voir les informations d'un exercice	M'informer sur le sujet d'un exercice
Utilisateur	Voir les informations d'un module	M'informer sur le sujet du module, l'objectif
Utilisateur	Voir les informations d'une évaluation	M'informer sur le sujet de l'évaluation, l'objectif
Utilisateur	Voir l'énoncé d'un exercice	D'apprendre une nouvelle notion, connaître le problème à résoudre, question à répondre pour valider la notion
Utilisateur	Proposer une solution à l'exercice	Gagner des PolyPoints et avancer dans le module associé
Utilisateur	Dans le cas d'un code à écrire, exécuter un validateur intermédiaire	Vérifier si mon code est correct pour le validateur en question
Utilisateur	Revoir la dernière solution qui à passée le plus de validateurs	Reprendre le code depuis un appareil différent, à un autre moment, pour l'améliorer
Utilisateur	Écrire (et modifier) sa solution de code dans un éditeur intégré à la page de l'exercice	Proposer une solution à l'exercice
Utilisateur	Ajouter des fichiers dans l'éditeur intégré à la page d'exercice	Organiser la solution en plusieurs fichiers
Utilisateur	Supprimer des fichiers dans l'éditeur	Organiser la solution en plusieurs fichiers
Utilisateur	Afficher les données de validateur (entrée et sortie) en échange de avec des PolyPoints	Comprendre mieux comment résoudre l'exercice
Utilisateur	Suivre ma progression dans chacun des modules	Voir ce qui est complété / à faire

En tant que…	Je veux…	Afin de…
Utilisateur	Voir le classement global des utilisateurs (par polypoints)	Nous motiver à atteindre le sommet (principe de concurrence)
Utilisateur	Passer une évaluation	Obtenir une certification
Utilisateur	Lire le contenu d'un cours	Monter en compétence sur un sujet
Créateur de contenu	Créer un exercice	Proposer l'apprentissage d'une nouvelle notion, faire vérifier la connaissance de cette notion par une question/ un code à écrire
Créateur de contenu	Créer un module	Organiser les exercices par notion majeure / thématique
Créateur d'évaluation	Créer une évaluation	Vérifier les compétence d'un utilisateur sur un contenu
Créateur de contenu	Ajouter ses exercices à un module qu'il a créé	Remplir le contenu d'un module en ensemble d'élément cohérent
Créateur de contenu	Ajouter des modules dans un module, et ce avec des modules qu'il a créé (sous-module)	Remplir le contenu d'un module en ensemble d'élément cohérent
Créateur de contenu	Modifier le nom, la description, le nombre de PolyPoints de récompense, les tags, le contenu (exercices et sous-module) de ses modules	Garder à jour un module
Créateur de contenu	Modifier le titre, la description, le contenu, récompense en polypoints, les validateurs, les tags d'un exercice	Garder à jour un exercice
Créateur de contenu	Modifier le titre, la description, le contenu d'une évaluation	Garder à jour une évaluation
Créateur de contenu	Supprimer un exercice qu'il a créé	Réparer une erreur / ne plus vouloir la présence de ce contenu
Créateur de contenu	Supprimer un module qu'il a créé	Réparer une erreur / ne plus vouloir la présence de ce contenu
Créateur de contenu	Supprimer une évaluation qu'il a créé	Réparer une erreur / ne plus vouloir la présence de ce contenu
Créateur de contenu	Voir le résultat des utilisateurs sur une évaluation qu'il a créé	Pour que le recruteur / professeur voie le résultat des élèves pour attribuer une note / recruter
Administrateur	Promouvoir un utilisateur en rédacteur	Qu'un utilisateur ai les droits d'un "redacteur"
Administrateur	Promouvoir un utilisateur en Administrateur	Qu'un utilisateur ai les droits d'un "Administrateur"

En tant que…	Je veux…	Afin de…
Administrateur	Créer un utilisateur	Utiliser l’application avec un autre compte
Administrateur	Récupérer les données d’un utilisateur	Voir les informations confidentielles d’un compte utilisateur
Administrateur	Mettre à jour les données d’un utilisateur	Mettre à jour les informations personnelles afin qu’elles soient cohérentes
Administrateur	Supprimer un utilisateur	Ne plus donner accès à la plateforme pour un compte utilisateur
Administrateur	Créer un exercice	Proposer l’apprentissage d’une nouvelle notion, faire vérifier la connaissance de cette notion par une question/ un code à écrire
Administrateur	Modifier le titre, la description, le contenu, récompense en polypoints, les validateurs, les tags d’un exercice	Garder à jour un exercice
Administrateur	Supprimer un exercice	Réparer une erreur / ne plus vouloir la présence de ce contenu
Administrateur	Créer un module	Créer un module afin de regrouper des contenus
Administrateur	Récupérer les données d’un module	Voir les informations et les contenus associés à ce module
Administrateur	Mettre à jour les données d’un module	Garde le module à jour
Administrateur	Supprimer un module	Effacer les traces du module sur la plateforme
Administrateur	Créer une évaluation	Vérifier les compétence d’un utilisateur sur un contenu
Administrateur	Récupérer les données d’une évaluation	Voir les différentes données en lien avec une évaluation
Administrateur	Mettre à jour les données d’une évaluation	Ajouter des utilisateurs ou modifier des données relatives à une évaluation
Administrateur	Supprimer une évaluation	Enlever une évaluation de la plateforme
Administrateur	Créer une team	Rassembler des utilisateurs dans une équipe
Administrateur	Ajouter un membre dans mon équipe	Proposer à un utilisateur de rejoindre mon équipe
Administrateur	Supprimer un membre d’une team	Enlever un utilisateur de mon équipe pour une quelconque raison

En tant que…	Je veux…	Afin de…
Administrateur	Supprimer une team	Supprimer une team qui ne valide pas les conditions d'utilisation
Administrateur	Modifier la description d'une équipe	Avoir une description à jour de l'équipe
Créateur d'évaluation	Créer une campagne de test	Evaluer le niveau des utilisateurs
Créateur d'évaluation	Ajouter des utilisateurs à ma campagne via une interface web	Faire participer les candidats
Créateur d'évaluation	Supprimer des utilisateurs à ma campagne via une interface web	Enlever un candidat des participants
Créateur d'évaluation	Ajouter des utilisateurs à ma campagne via des appels API	Faire participer les candidats
Créateur d'évaluation	Supprimer des utilisateurs à ma campagne via des appels API	Enlever un candidat des participants
Créateur d'évaluation	Ajouter des utilisateurs à ma campagne via l'importation de fichiers csv	Faire participer les candidats
Créateur d'évaluation	Voir les résultats et statistiques sur la campagne que j'ai créé	Me rendre compte du niveau des candidats testés
Créateur d'évaluation	Ajouter des tags à mes candidats	Grouper les candidats
Créateur d'évaluation	Définir une date limite pour ma campagne	Clôturer ma campagne à une date fixe
Candidat	Revenir sur un test et reprendre là où j'en était	Finir mon test si jamais je quitte l'application
Créateur d'évaluation	Définir un temps limite pour chaque question de ma campagne	Les candidats répondent dans un temps limité
Créateur d'évaluation	Définir un nb de points pour chaque question	Avoir un score par candidats et voir leur différence de score à la fin de la campagne
Candidat	Recevoir un mail me permettant de participer à une campagne de tests	Avoir un lien pour participer à une campagne
Candidat	Accepter de participer à une campagne	Tester ses compétences à travers une campagne
Candidat	Refuser de participer à une campagne	Avoir la possibilité de refuser une campagne et que le créateur en soit informé
Créateur d'évaluation	Éditer ma campagne, les tests liés	Modifier une campagne précédemment créée

En tant que…	Je veux…	Afin de…
Créateur d’évaluation	Définir une date de début de ma campagne	Définir une date pour les candidats, ainsi qu’un temps imparti pour finaliser la campagne
Créateur d’évaluation	Envoyer des liens de ma campagne manuellement à mes candidats	S’assurer que les candidats reçoivent bien le lien pour participer à une campagne
Candidat	Recevoir un mail de confirmation contenant des stats quand j’ai soumis mon test	Notifier l’utilisateur que sa participation et ses réponses ont bien été enregistrées pour une campagne
Créateur d’évaluation	Voir le nombre de points totaux par candidats	Comparer les points des candidats ayant participé à la campagne
Créateur d’évaluation	Visualiser un graphique/un excel par tags de content et par candidats	Voir graphiquement les différents résultats
Créateur d’évaluation	Exporter les resultats synthetisés dans un pdf	Sauvegarder les résultats des candidats et avoir une vue synthétique
Créateur d’évaluation	Exporter les resultats détaillés dans un pdf	Sauvegarder les résultats des candidats et y avoir accès sans passer par l’application
Créateur d’évaluation	Avoir une vue comparative des candidats sous la forme d’un tableau excel	Comparer les score des candidats à travers un tableau
Créateur d’évaluation	Trier la liste des candidats par tags, resultats	Comparer les résultats des candidats en fonction de données précises
Créateur d’évaluation	Télécharger les scores des candidats	Afin de garder les stats en local

Découpage en quartier fonctionnels

En considérant ces users stories on peut en déduire ces quartiers fonctionnels :

1. Authentification/Authorisation : permet à l’utilisateur de s’inscrire et de s’identifier sur la plateforme. Vérifie les droits de l’utilisateur sur une ressource.
2. Gestion des utilisateurs : permet la gestion des utilisateurs.
3. Edition de modules : donne la possibilité d’éditer et de visualiser des modules ainsi que leurs contenus et composants.
4. Envoi de mail : envoie des mail aux utilisateurs.
5. Gestion de campagne : donne la possibilité de la gestion des campagnes de tests.
6. Gestion des runners : permet de lancer des runners afin d’éxécuter du code.

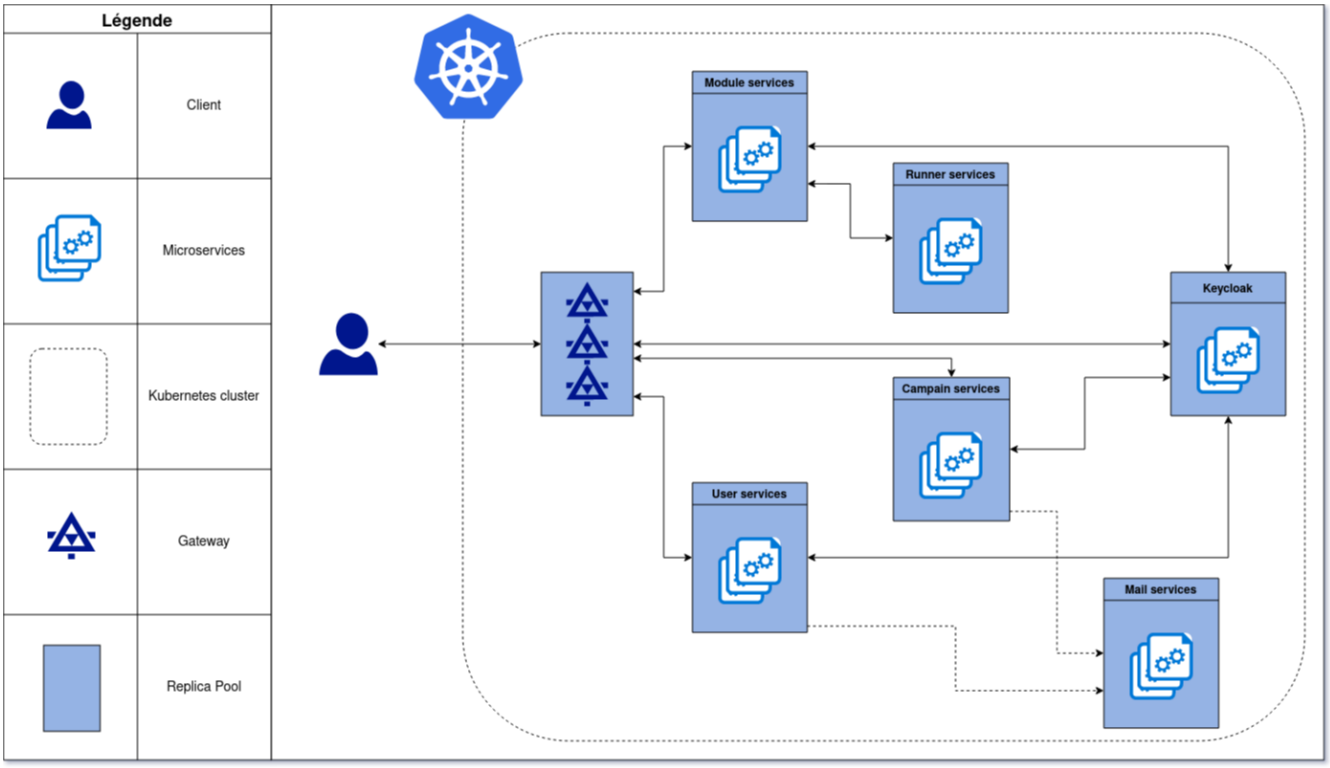


Figure 1. Architecture en microservices proposée

Justification de l'architecture

Le but premier de cette organsation est de réduire au maximum les dépendances entre chaque service notament au niveau des canneaux de communications, en effet on peut remarquer que peu de services comminiquent entre eux, hormis avec Keycloak qui est crucialpour l'idententification et l'autorisation (cf. à la question 2). Le fait de limiter le nombre de cannaux de communications permet de réduire les risques de défaillance générale et de faciliter la maintenance.

Le problème de ce choix technique est que l'on réduit certes les éventuelles erreurs inter-services mais en cas de panne de toutes les instances keycloak, les utilisateurs ne pourront pas interagir avec l'application.

Vu que nous nous plaçons dans une architecture de microservices il faut partir du principe que nos services auront des réplicas qui vont crasher et dautres qui vont démarrer. Pour assurer un bon routage de nos requêtes, nous allons utiliser un service registery afin de garder en mémoire nos services encore vivants ainsi que leur adresse IP (cf. Question 9 - Enregistrement des runners).

Il à aussi été envisagé de faire un service *Équipe* et *Contenu* séparés mais cela aurait impliqué de faire des appels API supplémentaires et donc d'encore augmenter le nombre de cannaux de communications et donc d'augmenter le temps de latence. De plus ça n'aurait aucun sens car les notions d'utilisateur et d'équipe sont interdépendantes et qu'un module n'a au final d'intéréssant que les contenus qu'il contient.

Q2: Comment gérer le système d’authentification avec un OIDC?

Q3: Communication Inter micro-services

Quand nous regardons les microservices que nous avons défini plus tôt nous pouvons remarquer que grand nombre d'entre eux devront implémenter un CRUD tel que le microservice des utilisateurs, des modules etc., il est donc nécessaire d'utiliser un protocole de communication synchrone afin de faire remonter une erreur en cas de problème au niveau des bases de données. Pour ce faire ces services vont utiliser le protocole HTTP avec des endpoint REST.

Pour les microservices qui ne sont pas des CRUD comme le microservice d'envoi de mail, il est plus intéressant d'utiliser un protocole de communication asynchrone. En effet, si nous prenons toujours l'example de l'envoi de mail, il est inutile de faire remonter une erreur au niveau de l'API gateway si le mail n'a pas pu être envoyé, ce dernier pouvant être renvoyé, il est préférable de simplement logger l'erreur et de continuer le traitement.

Dans des circonstances normale nous serions partis sur un système de message queue tel que RabbitMQ, système grâce auquel toutes les communications sont stockées dans une file d'attente permettant au destinataire de traiter les messages à son rythme et de reprendre là où il en était en cas de crash. Cependant le sujet nous interdit explicitement d'utiliser ce type de protocole.

Dans un premier temps il a été envisagé d'utiliser la technique du http long polling. Cette technique consiste à faire de longues requêtes HTTP en boucle jusqu'à ce que le serveur renvoie une réponse, ce qui permet de simuler une communication asynchrone. Dans notre cas c'est le microservice d'envoi de mail qui va faire les requêtes HTTP jusqu'à ce qu'une réponse soit reçue. Cette approche est très simple conceptuellement, dependant elle présente beaucoup trop de désavantages pour être utilisée. En effet un *proof of concept* [poc_q3_lp] ainsi que le document RFC6202 "Known Issues and Best Practices for the Use of Long Polling and Streaming in Bidirectional HTTP" [rfc6202] publié en avril 2011, mettent en évidence de nombreux problèmes liés à cette technique dont les plus importants sont:

- Une consommation importante de ressources côté serveur, en effet le serveur doit maintenir une connection ouverte avec le client et doit traiter les requêtes HTTP même si aucune réponse n'est attendue.
- Une certaine latence, même si l'envoi de mail est un processus long, il est préférable de réduire la latence de nos processus au maximum.
- Les timeout pouvant poser problème dans la mesure ou les proxys peuvent fermer la connexion avant que le serveur ne renvoie une réponse.

Suite à un deuxième *proof of concept* [poc_q3_ws], une autre solution est d'utiliser des websockets pour communiquer avec ces services, en effet les websockets permettent des communications asynchrones entre nos services qui peuvent passer à travers un proxy. Hélas cette solution admet deux gros problèmes :

- Il est impossible de pour l'un des deux parties de savoir si l'autre est indisponible ce qui pose problème dans un environnement en picroservice où tout service peut s'arrêter à tout moment. Une implémentation pour relancer le socket s'impose donc.
- Les connections étant persistentes, un scaling horizontal est tout bonnement impossible, les deux parties devant absolument être les mêmes.

La solution la plus simple et posant le moins de soucis reste des appels HTTP vers une API REST que notre fil d'éxécution principal n'attendra pas, même si cette solution est moins *fault tolerant* qu'une message queue, elle a le mérite d'être très simple à impémenter (cf. [poc_q3_http]), là ou du RPC nécéssiterait des fichiers de définition. De plus cela permet d'éviter de faire de nos microservice des hybrides API REST/Serveur long polling ou API REST/Serveur websocket comme vous pouvez le voir dans les premiers *proof of concept* [poc_q3_ws] [poc_q3_lp].

Pour illustrer notre solution faisons un diagramme de séquence en prenant l'exemple de quelqu'un s'incrivant à Polycode:

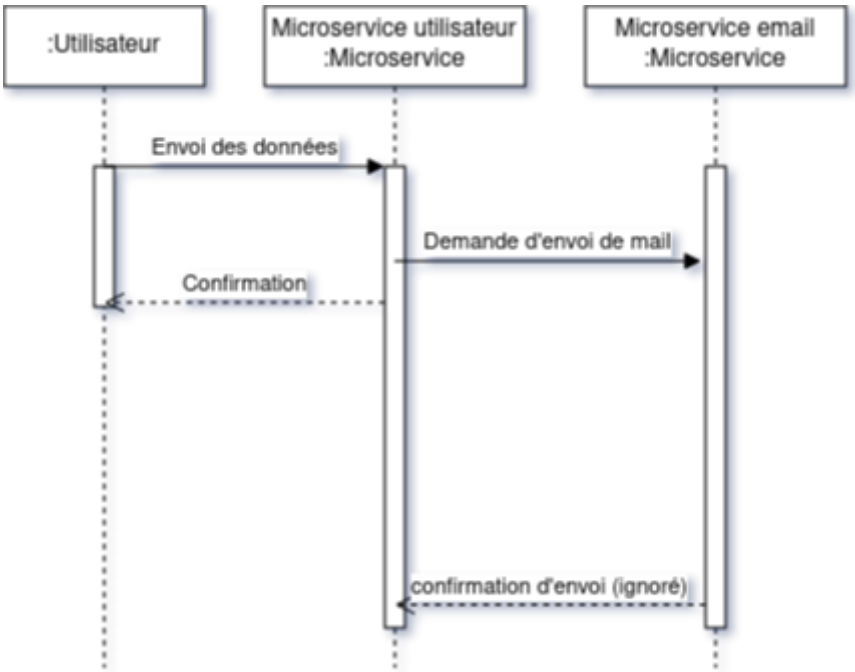


Figure 2. Diagramme de séquence d'une inscription

Q4: Comment tracer les logs et les requêtes?

Le traçage distribué est une méthode employée pour suivre le parcours d'une requête dans un système distribué comme dans le cas d'une architecure en microservices. Il permet de suivre les requêtes et les réponses entre les différents services.

Une solution de traçage distribué va marquer une requête de l'utilisateur avec un identifiant unique et le transmettre à chaque service qui la reçoit. Chaque traitement effectué sur la requête va ajouter des informations tel que le nom du service, le temps de traitement, etc. Zipkin est une solution qui va collecter les informations de traçage et de présenter les données de façon compréhensible. Zipkin est composé de 3 composants :

- Un serveur qui va collecter les informations de traçage et les stocker dans une base de données.
- Un client qui va ajouter des informations de traçage à chaque requête.
- Une interface web qui va permettre de visualiser les informations de traçage.

On peut associer une base de données au serveur Zipkin pour stocker les informations de traçage.

Voici l'architecture proposée pour l'intégration de Zipkin dans notre application :

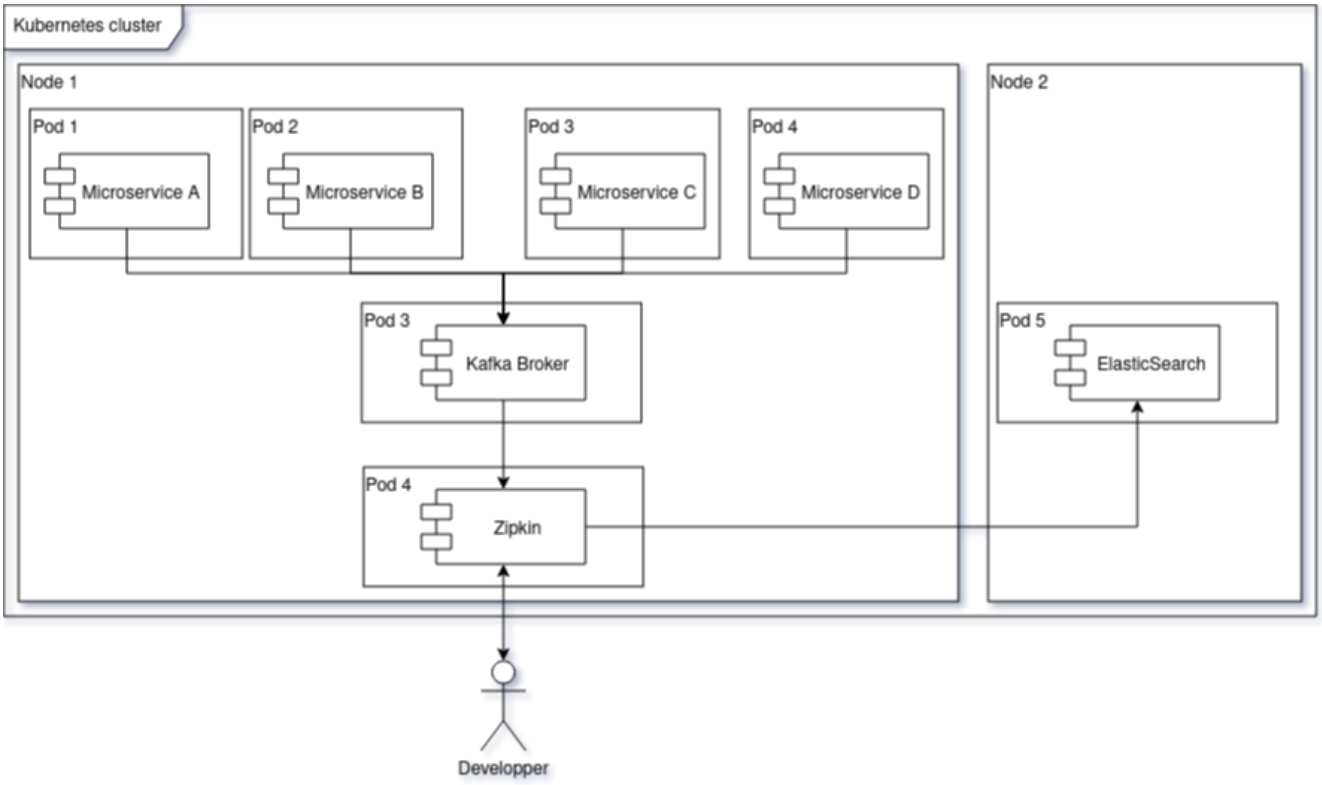


Figure 3. Proposition d'architecture pour l'intégration de Zipkin

Nous aurions pu utiliser Jaeger à la place de Zipkin, cependant Zipkin supporte plus de langages.

Les logs seront stockés dans une base de données Elasticsearch, ses performances permettent le stockage et la lecture de nombreux logs. Malgré son efficacité nous prendrons bien soin de la placer sur un autre noeud afin que ses opérations ne perturbent pas d'autres pods.

Voici un diagramme de séquence qui illustre le fonctionnement de Zipkin et d'Elasticsearch lors

d’une création de compte :

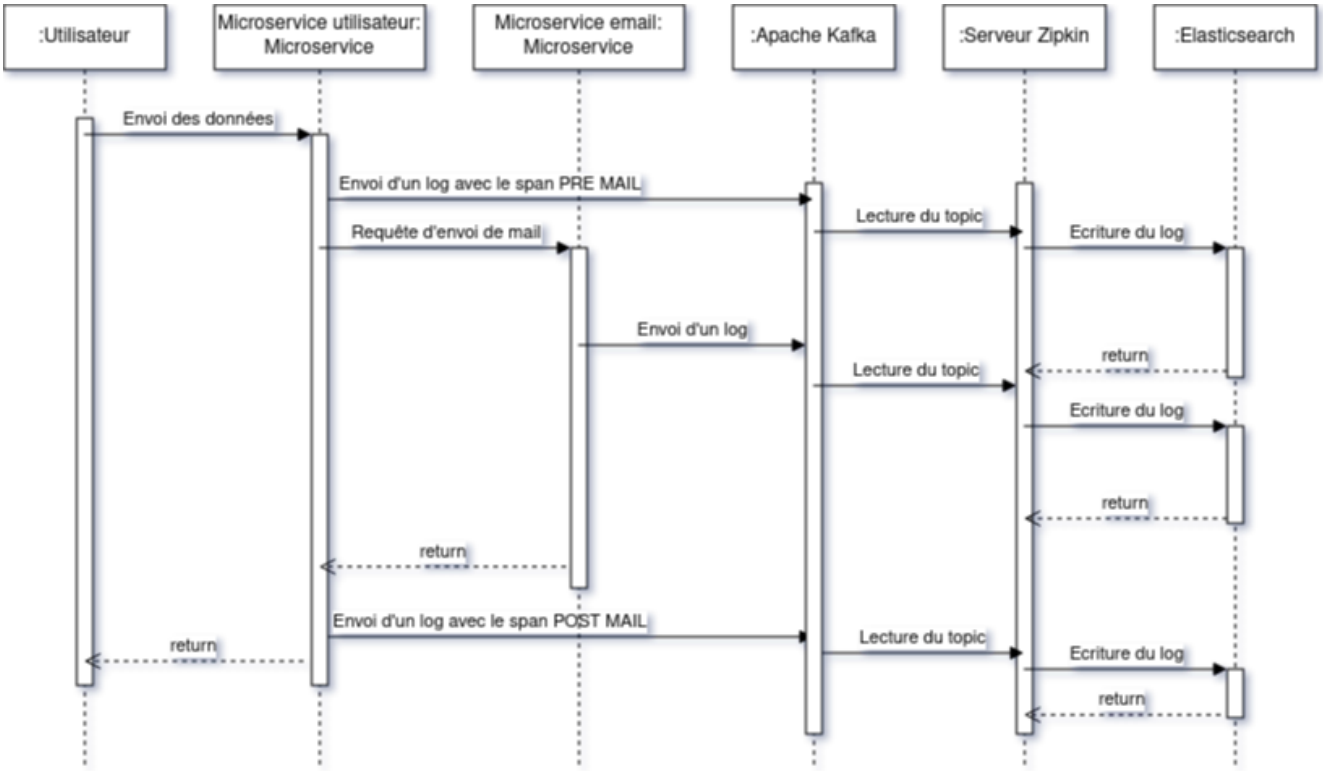


Figure 4. Diagramme de séquence décrivant la création de compte

Apache Kafka est utilisé dans cette configuration pour envoyer les logs à Zipkin, le but de cette utilisation est d’envoyer des requêtes de manière asynchrone afin de ne pas ralentir les microservices utilisateur et d’envoi de mail, ainsi une inscription sera rapide pour l’utilisateur.

Q5: Comment ajouter un moteur de recherche?

Moteur de recherche

Afin de pouvoir rechercher du texte grâce à une barre de recherche, nous devons d'abord identifier les données pouvant être sujets à une recherche. À l'heure actuelle nous souhaitons pouvoir retrouver un exercice ou un cours à partir du texte qu'il contient, à l'heure actuelle les cours de Polycode sont organisés avec cette structure.

Classes impliquées dans la recherche

```
class Module {
    id: uuid;
    name: string; // Champ sujet à recherche
    description: string; // Champ sujet à recherche
    type: 'challenge' | 'practice' | 'certification' | 'submodule' | ...;
    // ...
    contents: Content[];
    modules: Module[];
    tags: string[]; // Champ sujet à recherche
}

// ...

class Content {
    id: uuid;
    name: string; // Champ sujet à recherche
    description: string; // Champ sujet à recherche
    type: 'exercise' | 'lesson' | ...;
    rootComponent: Component;
    // ...
}

// ...

class Component {
    id: uuid;
    type: 'container' | 'editor' | 'quizz' | 'markdown';
    // ...
    components: Component[];
    markdown: string; // Champ sujet à recherche
}
```

Les champs name, description, tags et markdown sont des champs sur lesquels l'utilisateur pourra effectuer des recherches full text search. Cependant pour éviter d'avoir à indexer plusieurs champs d'un même document, le microservice module se chargera d'insérer un document dans une autre

base de données contenant la concaténation de ces champs et de l'identifiant de l'objet en question. Ce même microservice se chargera aussi d'effectuer les recherches.

Structure du document dans la base de données de recherche

```
type IndexedDocument = {
    id: uuid;
    text: string; // le texte concaténé

    // ces champs ne seront jamais utilisés pour une recherche
    data : {
        type: 'module' | 'content' | 'component';
        name: string;
        description: string;
    };
};
```

La base de données en question sera Elasticsearch, cette dernière utilisant le moteur apache Lucene, il nous sera possible de faire des recherches en full text search sur le texte de chaque module, contenu et composant. Tout comme pour les logs nous placrons cette base de données sur une autre machine physique, séparée des autres services pour ne pas interférer avec eux.

En cas de nouvelles attentes concernant la recherche (par exemple la recherche d'utilisateurs), il serait envisageable de créer un microservice dédié à la recherche. Nous ne le créerons pas pour l'instant car seuls les modules, contenus et composants sont concernés par la recherche et que cela violerait le principe YAGNI (You Ain't Gonna Need It)

Nous avons donc ces diagrammes de séquence :

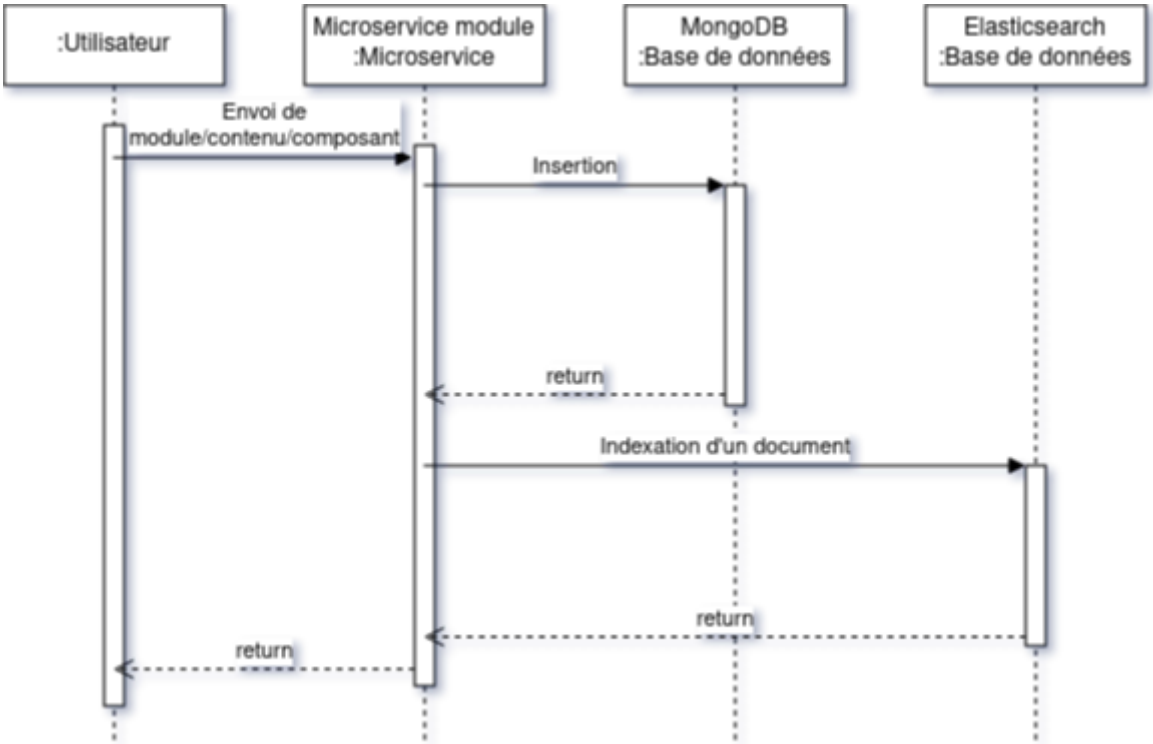


Figure 5. Diagramme de séquence décrivant l'indexation d'un module

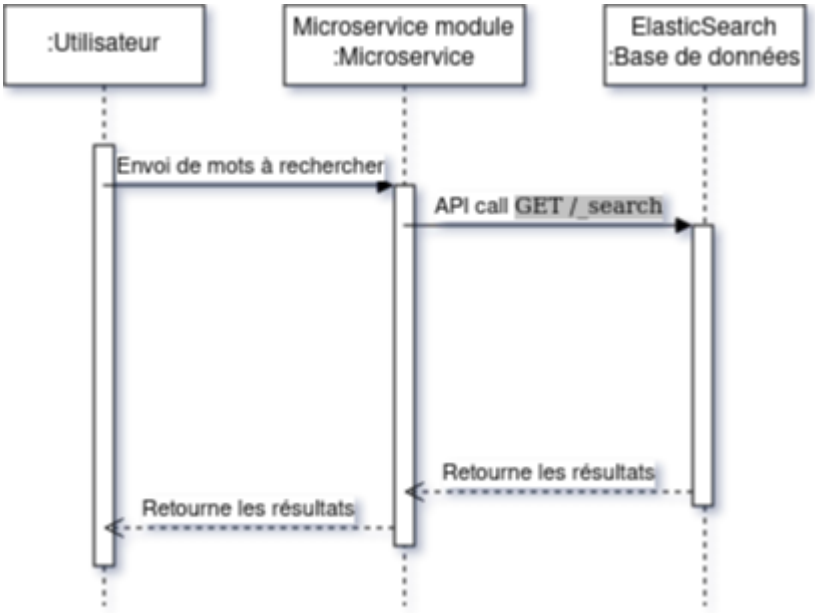


Figure 6. Diagramme de séquence décrivant la recherche par mot clé

Intégration dans le frontend

Concernant le frontend nous pouvons imaginer une barre de recherche dans l'en-tête de la page. Qui une fois clickée ouvrira une fenêtre modale avec un champ de recherche. Cela permettra de pouvoir rechercher sans avoir à se soucier des composants de la page.

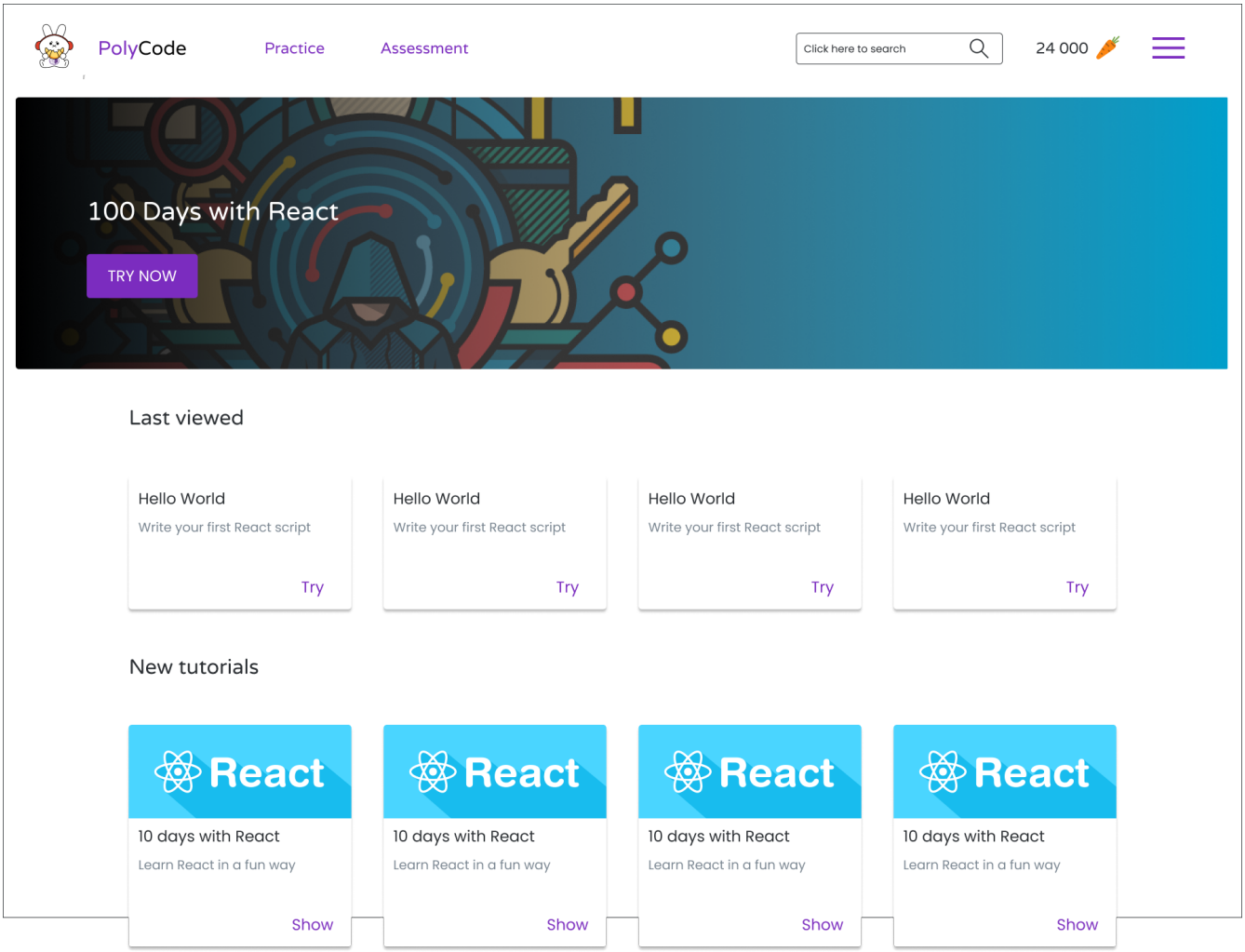


Figure 7. Proposition d'intégration de la barre de recherche

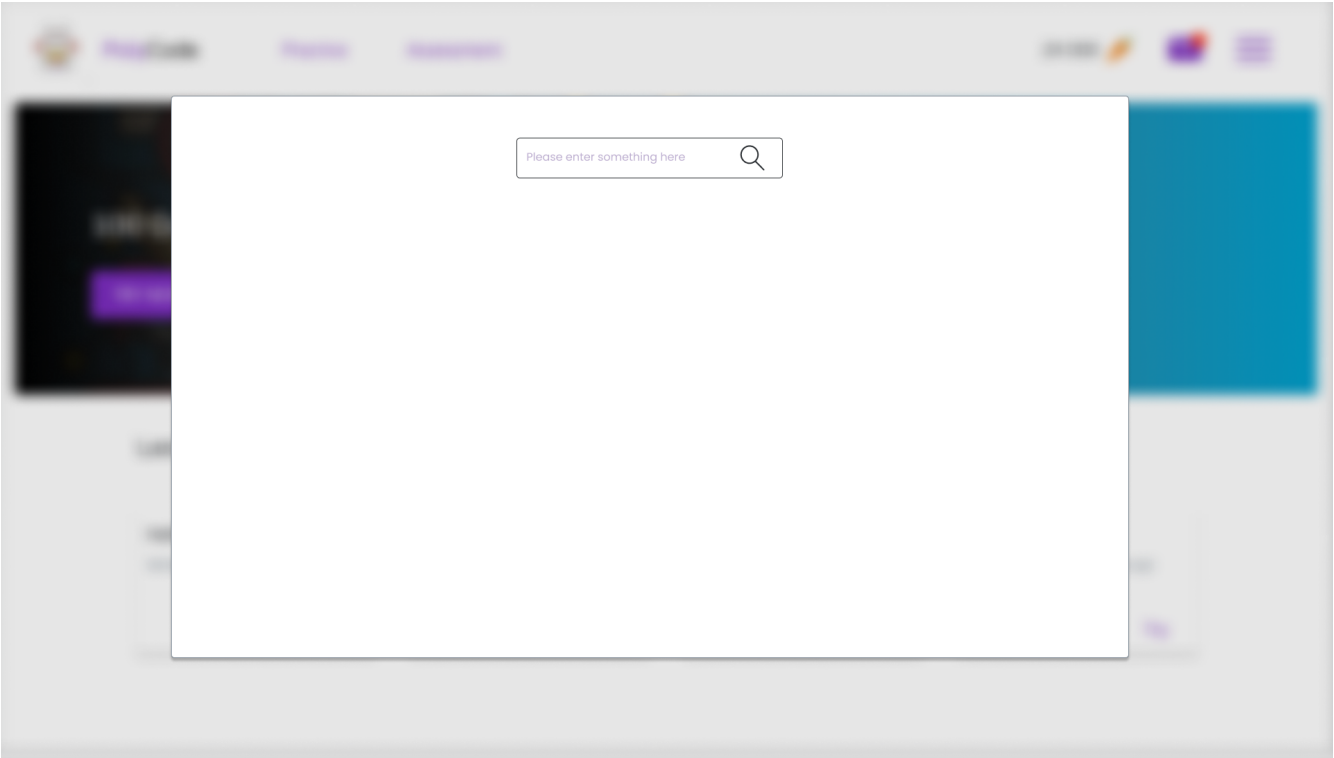


Figure 8. Proposition d'interface de recherche

Quand l'utilisateur rentre du texte une requête est envoyée au microservice de modules qui va renvoyer les résultats. Les données contenues dans le champ `IndexedDocument.data` permettront de

détailler les résultats de la recherche.

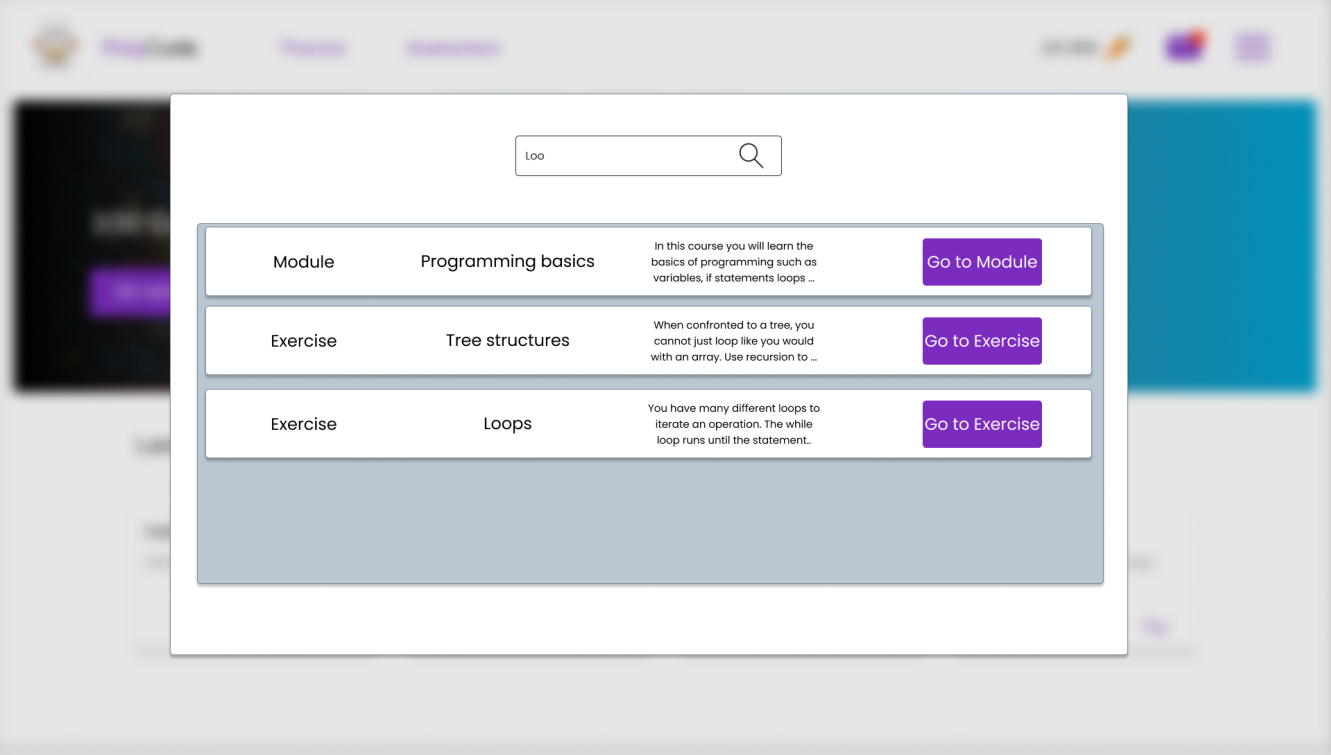


Figure 9. Proposition d'interface de résultats de recherche

Q6: Comment mettre en place l'architecture de runner

Définition du runner

Le runner est un élément central dans le fonctionnement de Polycode. Il s'agit d'une API à laquelle on donne du code, un language et des paramètres à passer par l'entrée standard, elle est chargée de créer des environnements isolés (conteneurs, machines virtuelles etc) qui vont se charger d'exécuter le code en renvoyant le résultat de l'exécution sur leur sortie standard. C'est cette sortie standard qui sera comparée aux données du validateur pour déterminer si l'exercice a été réussi ou non.

Architecture

La première chose à laquelle on peut penser en parlant d'isolation c'est de privilégier la création de machines virtuelles qui, contrairement à ce que l'on pense, peuvent démarrer relativement rapidement. Ce besoin de d'isolation est important car il faut éviter que le code de l'utilisateur puisse interférer avec l' infrastructure avec d'autres processus, isolation que Docker ne peut pas garantir car les conteneurs docker sont des procesuss isolés avec `iptables` et `cgroup`.

En plus d'une isolation du code, il faut aussi isoler le réseau pour empêcher l'utilisateur d'utiliser du code qui pourrait cibler nos services internes, c'est pour cela que nous empêcherons les runners d'envoyer des paquets réseau via un pare feu.

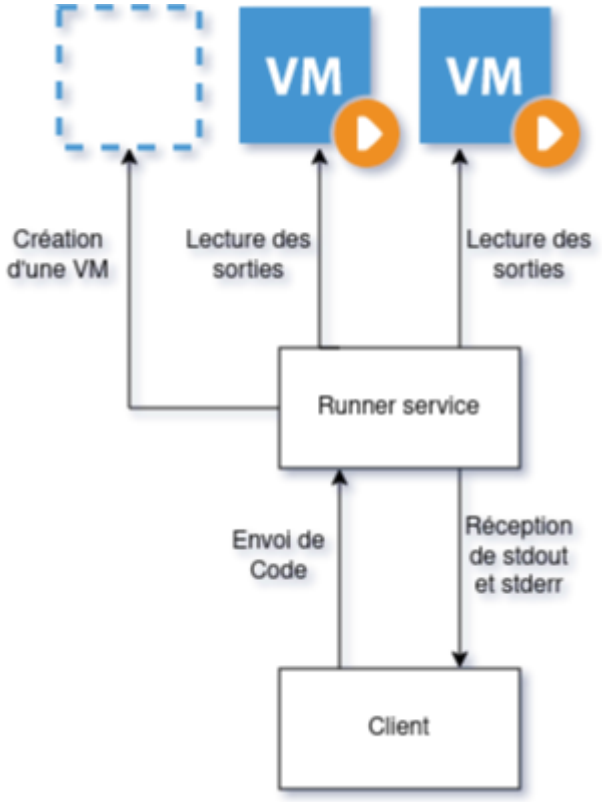


Figure 10. Schématisation du service de runner

L'équipe de Polycode est en ce moment même en train de développer un gestionnaire de machine

virtuelle qui gèrera ces aspects d'isolation et de lancement de machines virtuelle.

Enregistrement des runners

Afin de pouvoir contacter les runners pouvant être sur plusieurs machines virtuelles nous aurons besoin d'un register qui gardera en mémoire l'adresse des différents services. Nous utiliserons Consul par HashiCorp qui permet de faire de la découverte de service, les runners s'enregistreront auprès de consul et consul nous permettra de les récupérer, l'avantage de Consul est que les proxy qu'il génère supportent de base la répartition de charge, il sera donc plus facile de distribuer les services runners sur plusieurs machines. Le même raisonnement s'applique sur nos autres microservices.

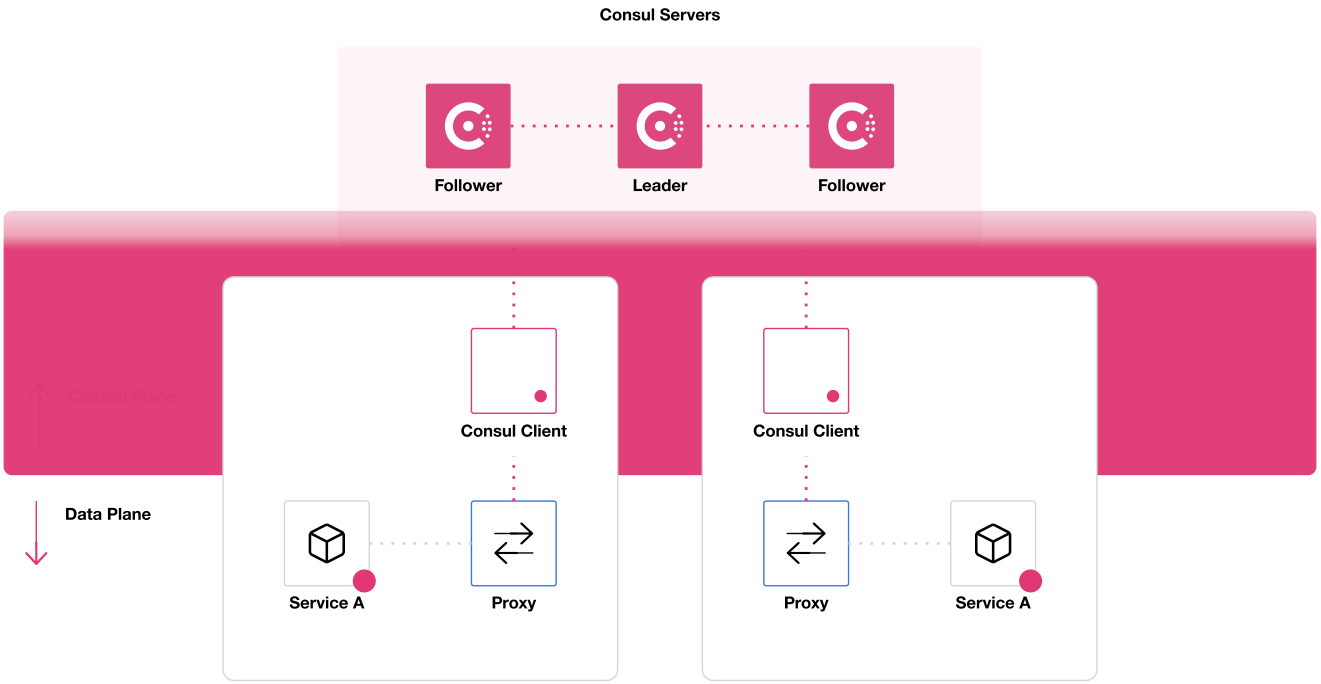


Figure 11. Schéma issu du site internet de Consul

Q7: Comment gérer les données?

Organisation des bases de données

Quand on se pose la question de l'organisation des données la première question que l'on peut se poser est "comment vont être associés les services et les bases de données ?", pour répondre à cette question on a deux grandes approches: Le pattern "shared database" et le pattern du "Database per service".

La première aproche consiste à mettre en place une de base de données (ou un cluster) dans laquelle tous les services viendraient écrire et lire. Cette manière d'organiser sa base de donnée permet d'avoir des services qui sont aisément capables de faire des jointures si besoin, la source de donnée étant unique.

Cependant ce choix de design n'est adapté qu'à des architectures monolithiques, il est même considéré par beaucoup comme un anti-pattern. En effet la base de données consistera en un *Single point of failure* si la base de données plante, l'entièreté du système sera paralysé. De plus cela entrave le développement des services car tous les services seront développés pour un seul type de base de données. Ces pour ces raisons que nous rejetons instantanément ce pattern.

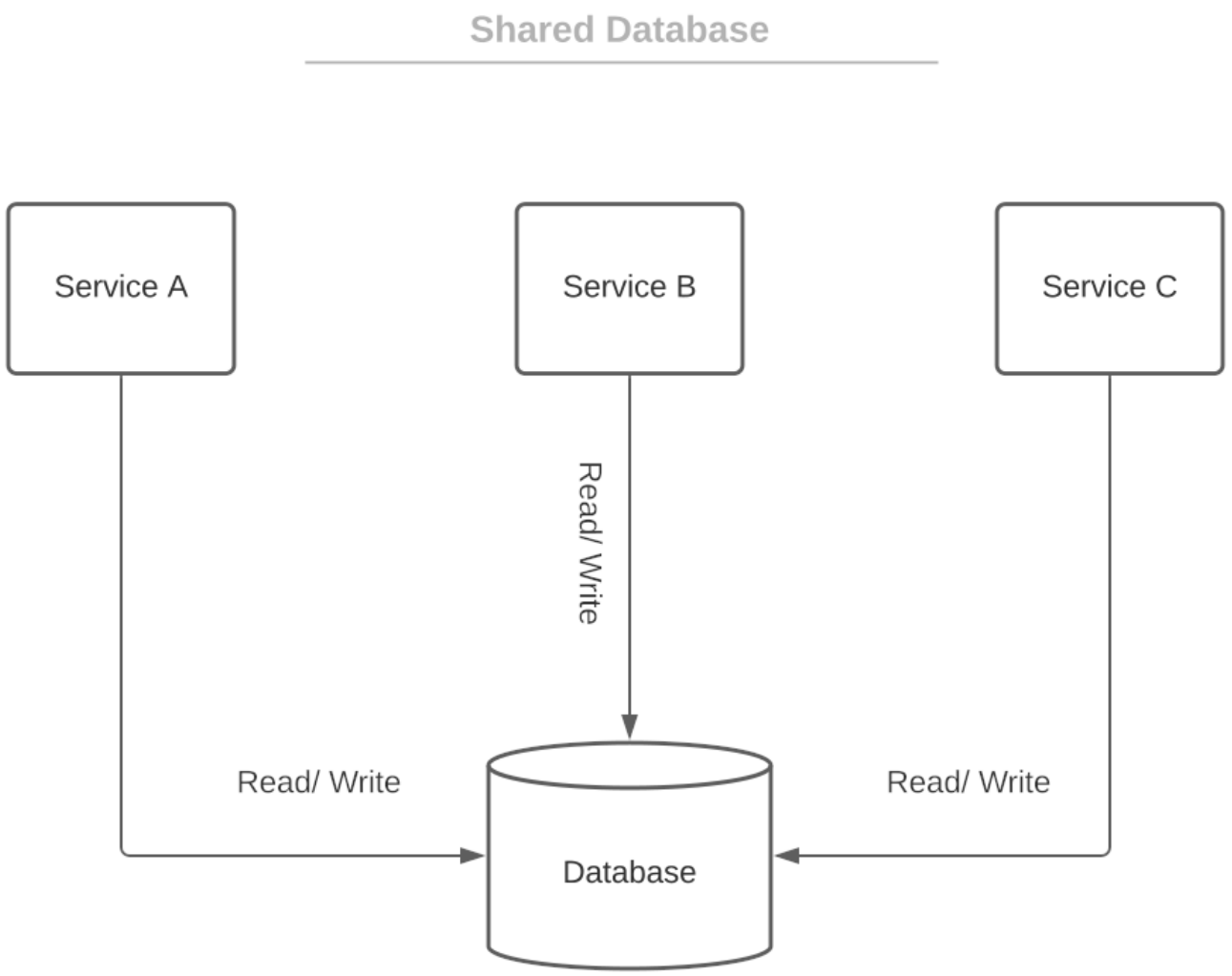


Figure 12. Shared database pattern

À l'exact opposé nous avons le pattern *database per service* dans lequel les instances de chaque service ont à disposition une base de données dédié, inaccessible par d'autres services. Ce découplage entre les services permet d'éviter de surcharger une seule base de données et donc réduit les latences de chaque service. De plus cela permet d'avoir une application Polyglotte, c'est à dire que grâce à cette separation des services vis à vis de leurs données, nous pourrons avoir des services utilisants différents SGBD en fonction de nos besoins.

Au vu des avantages du pattern *database per service*, nous allons l'implémenter dans notre architecture, à la seule différence que nos services seront reliés à des clusters de base de données, ce qui permettra de répartir la charge et d'avoir des données cohérentes.

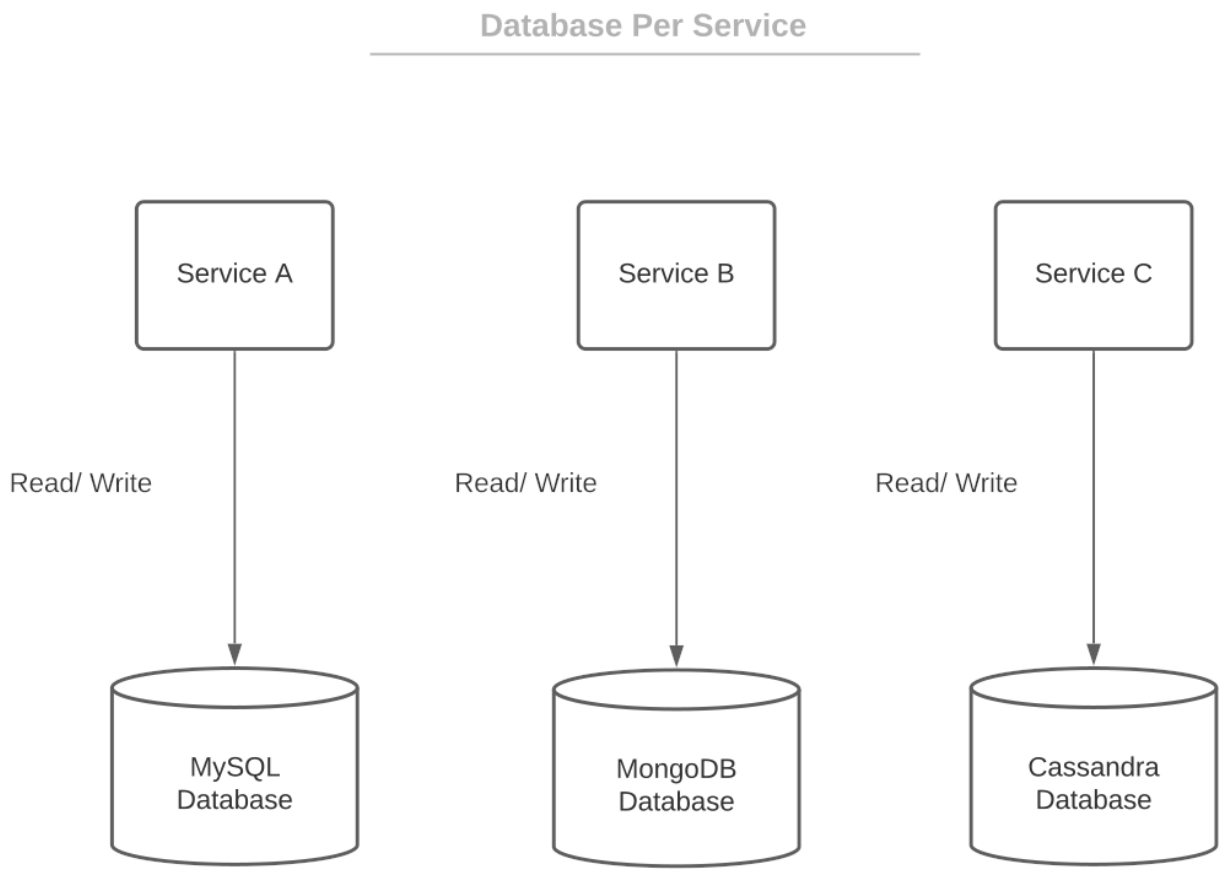


Figure 13. Database per service pattern

SGBD utilisés

Nos services utiliserons différents types de base de données en fonctions de nos services:

- Gestion des utilisateurs : Relationelle. Cette base de données stockera les utilisaturs, que leurs adresses email, leurs paramètres ainsi que les transactions effectuées en Polypoints. Nous voulons nous assurer que les données soient ACID car les informations stockées (notament les adresses mails, les mots de passe ou encore les items achetés) sont assez importantes pour éxiger une cohérence forte.
- Edition de modules : Documentaire. Les modules, composants, validateurs et composants ont été désignés pour être des éléments centraux et réutilisable de Polycode remplissent de nombreuses fonctions en effet ils sont utilisés dans le cadre des exercices, des évaluations et

dans les campagnes de test. Cette variété implique nécéssairement des schémas qui vont varier. Les bases de données documentaires sont adaptées à ce genre de cas de part l'abscence de schéma précis prédéfini.

- Gestion de campagne de tests: Relationelle. Les campagnes de tests sont des structures de données auquelles sont rattachées les candidats ainsi que des tags. Nous souhaitons des données les plus cohérentes car il serait malheureux que des utilisateurs ne soint pas reliés à une campagne à cause d'un manque de cohérence.

```
## Campaign
- id: `uuid`
- name: `string`
- description: `string`
- testId: `uuid`
- creatorId: `uuid`

# ...

## Candidate
- id: `uuid`
- campaignId: `uuid`
- userId: `uuid`
- status: `string{Confirmation Pending, Confirmed, Declined, Cancelled, Test Sent,
Test Opened, Test In Process, Test Finished}`
- startedAt: `Date`
- email: `string`
- linkCode: `uuid`
- linkExpiration: `Date`
```

Pour les bases de données relationnelles nous utiliserons PostgresSQL. En effet ce dernier comparé à d'autres SGBD s'est imposé par ses nombreuses fonctionnalités et ses languages supportés malgré le fait qu'elle ne soit pas la plus rapide. Cependent le réel intérêt de postgres c'est sa fiabilité quant à l'écxécution des transactions. En effet PostgreSQL tient un Write Ahead LOG (WAL), soit des logs écrits avant les transactions, ce qui permet de les re-éxécuter si la transaction s'arrête inopinément comme par exemple dans le cas d'une panne. Ce journal peut être envoyé à d'autres instances pour former un cluster, le transfert peut se faire en streaming ou via FTP pour privilégier soit respectivement pour une réplication asynchone ou une réplication synchrone.

Dans un cluster postgres nous avons deux types de noeuds: les noeuds maîtres qui se chargent des opérations d'écriture faisant preuve de source de vérité, et les noeuds esclave qui eux se contentent de copier les données des maîtres afin d'être opérationnels pour les opérations de lecture.

-

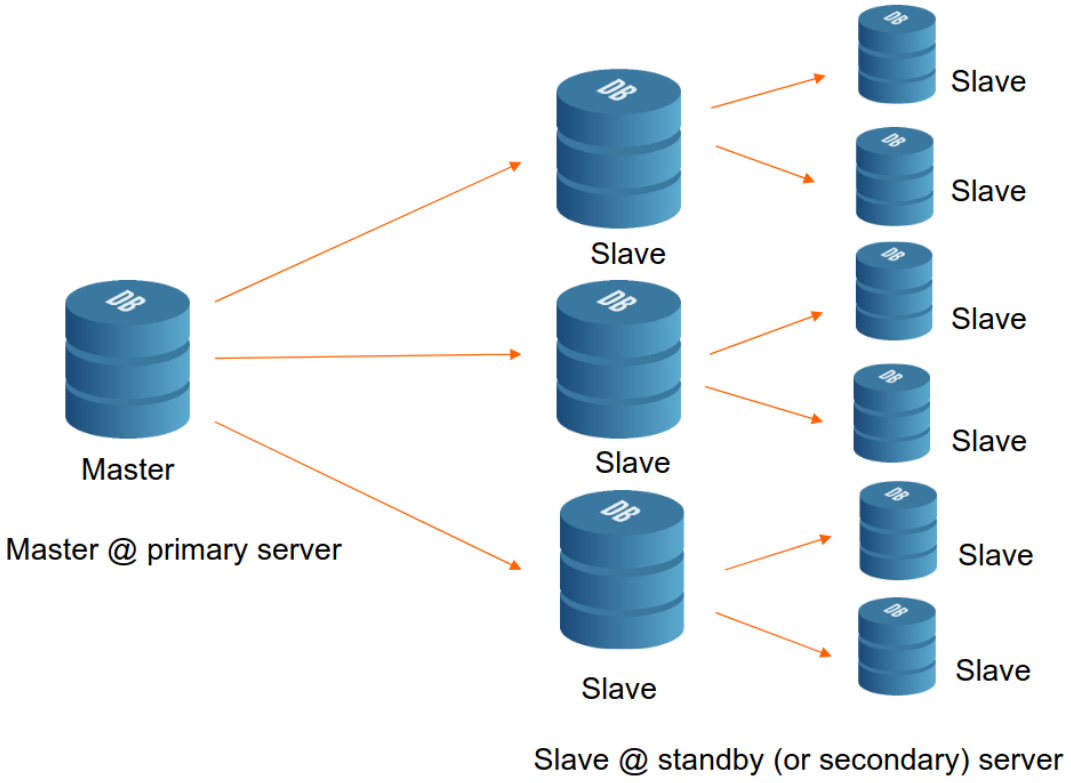


Figure 14. Exemple de cluster Postgres

Pour ce qui est de l'organisation du cluster nous avons deux choix:

- Avoir plusieurs noeud maîtres, ce qui permet une plus grande disponibilité et une bonne tolérance aux pannes quand à l'écriture des données en effet si un des noeuds maîtres tombe en panne, un autre peut prendre la relève. Cependant la présence de multiple sources de vérité peut amener à des conflits qui doivent être résolus.
- Avoir un seul noeud maître. Cette approche propose l'exact inverse de l'approche multi-maître: à savoir une abscence de conflits mais une moins bonne disponibilité.

Nous souhaitons à tout prix avoir des bases de données hautement disponibles mais avec les données les plus cohérentes possibles. Nous allons donc répondre au premier besoin avec un cluster multi-maître, et au second point en utilisant le streaming du WAL, afin que les données soient mises à jour le plus fréquemment possible pour éviter des conflits.

Alternativement, il a été envisagé d'utiliser Apache Cassandra, une base de données orientée colonnes, souvent utilisée pour rapidement stocker de grands volumes de données, en effet chaque noeud d'un Ring est capable de contenir juqu'à plus de 1 Teraoctet de données et faire énormément d'écritures par seconde. Néanmoins, au delà du fait Cassandra n'est pas une base de données *ACID compliant* de part l'abscence de transactions et de transactions, il y a aussi une bascence de jointure ce qui implique donc un refactoring de nos données mais aussi des données dupliquées pour chacune de nos relations n-aires ce qui peut être compliqué à gérer. Nous n'avons donc pas gardé cette solution.

Pour ce qui est des bases de données orientée documents 3 noms sont ressortis pendant mes recherches: CouchDB, CouchBase et MongoDB. Tous deux sont des bases de données orientées document très similaires mais ont quand même beaucoup de différences.

CouchDB est un projet de la fondation Apache qui vise à fournir une base de données orientée document, vouée à être acessible et manipulable via une API REST mais aussi à être ACID. Malgré cet avantage il y a des inconvéniants majeurs comme le fait que les données soient stockées en format JSON (JavaScript Object Notation) qui est un format lourd, le fait que ce ne soit pas stocké en mémoire, l'abscence de transactions la latence observée sur les grands sets de données. Nous rejeterons donc ce SGBD pour ces raisons.

Pour pallier à une partie de ces problèmes il existe Couchbase, un SGBD créé par la fusion de CouchOne, une base de données basée la base de données orientée document couchDB, et de membase une base de donnée clé-valeur stockée en mémoire. Elle a la particulité de stocker ses documents sur disque, mais aussi de placer les plus récents dans la mémoire RAM.

La solution pour laquelle nous avions optée dans les précédentes versions de Polycode est MongoDB. Il s'agit d'un SGBD orienté document dont la princiale particularité est que le stockage des données est en BSON, qui un format de données binaire qui permet de stocker des données JSON de manière plus compacte.

MongoDB et couchbase ayant beaucoup de points communs nous allons les comparer dans le tableau suivant:

	MongoDB	**CouchBase**
Indexage	Indexage par arbre binaire. Temps moyen `O(log n)`, temps dans le pire des cas `O(log n)`	Indexage par skip list. Temps moyen `O(log n)`, temps dans le pire des cas `O(n)`
Performances	Performances constantes peut importe la taille du cluster	Performances limitées dans un petit cluster mais augmentent rapidement avec la taille du cluster
Stockage des données	Stockage sur disque avec le format BSON	Les documents les plus récents ont droit à une copie en mémoire, les documents sont stockés dans des *virtuals Buckets* qui sont copiés sur plusieurs noeuds d'un cluster

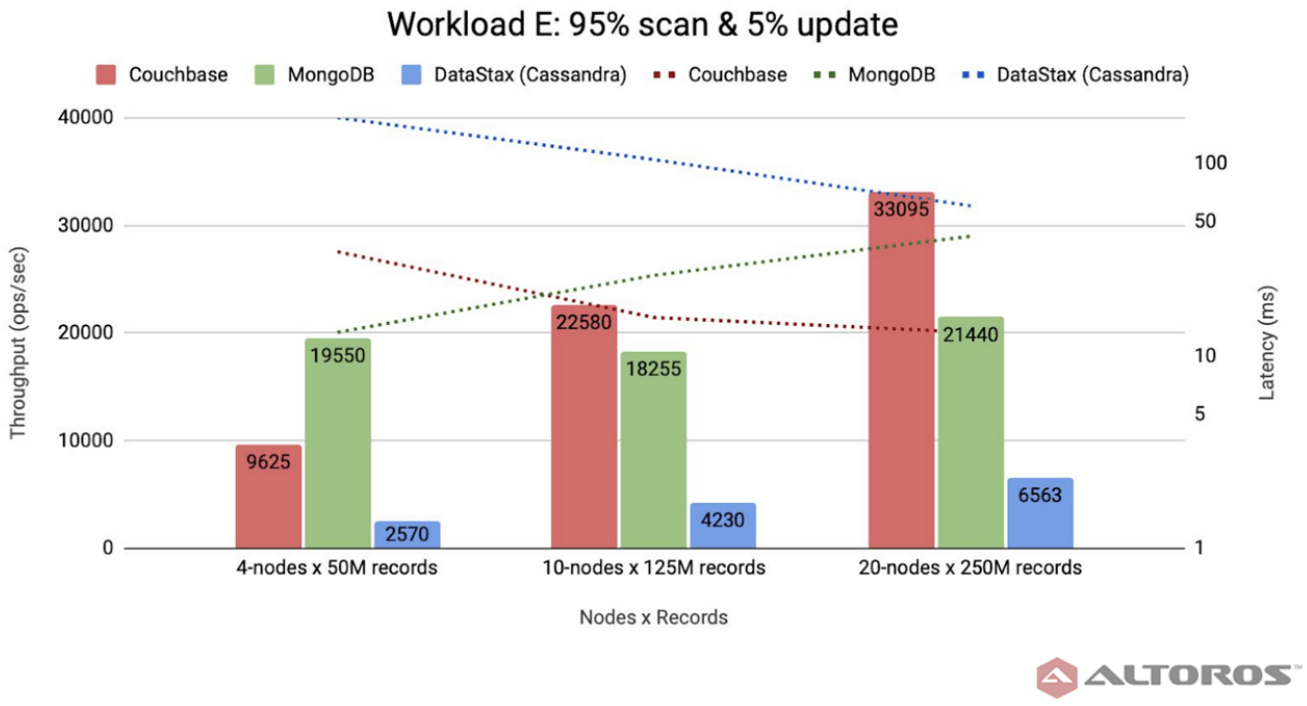


Figure 15. Comparaison entre CouchBase MongoDB et Cassandra

Concrètement couchbase est plus appropriée dans des environnements de grande taille, tandis que MongoDB est plus Polyvalent. Nous avons donc opté pour MongoDB de part cette polyvalence.

Q8: Comment ajouter une application mobile dans le système

Fonctionalités et mockup visuels

Dans le cadre d'une application mobile intégrée à Polycode, nous souhaiterions proposer une expérience la plus proche possible que ce que l'utilisateur peut avoir sur un ordinateur, tout en l'adaptant le plus possible au support Android.

C'est autour de cette ligne directrice que nous avons défini les fonctionnalités que nous souhaiterions implémenter dans notre application mobile.

Premièrement des fonctionnalités hors ligne seront proposées:

- Les informations de l'utilisateur notament le nom d'utilisateur, ses adresses email, son nombre de polypoint, ses équipes et son rang seront stockées en local et synchronisées avec le serveur lorsqu'une connexion internet sera disponible.
- À chaque connexion à internet l'application va garder en cache une partie des modules et des contenus chargés afin de pouvoir les lire et de les compléter hors ligne.
- Le code écrit dans un éditeur de code par l'utilisateur sera stocké en local, afin d'être modifié même hors ligne.
- Un utilisateur pourra soumettre du code et des réponses aux exercices même hors ligne, l'application s'occupera d'envoyer les données lorsqu'une connexion internet sera disponible et notiefira l'utilisateur de ses résultats.

Comme sous entendu ci-dessus nous autorisons l'utilisateur à répondre aux contenus et nottaments ceux de type éditeur de code sur téléphone. Pour faciliter la saisie de code, il sera mis en place un éditeur de code avec coloration syntaxique, autocomplétion ainsi qu'une barre horizontale placée au dessus du clavier de l'utilisateur, sur laquelle sera placée des characatères spéciaux fréquemment utilisés dans le language de programmation contenu dans l'éditeur. En cliquant sur ses caractères spéciaux, ils seront automatiquement ajoutés à la position du curseur dans l'éditeur pour que l'utilisateur n'aie pas a naviguer dans les symboles de son clavier, tout comme le shell pour Android Termux.

Editeur de code sans focus du clavier

9:41

Back

```
class MyClass {
  static fileExtentions = new Map ([

    [".csv", CSVQueryGenerator],
    [".ttl", TurtleQueryGenerator],
    [".turtle", TurtleQueryGenerator],
    [".rdf", RDFXMLQueryGenerator],
    [".rdfxml", RDFXMLQueryGenerator],
    [".owl", RDFXMLQueryGenerator],
    [".jsonld", JSONLDQueryGenerator],
    [".json", JSONLDQueryGenerator]
  ])

  constructor (filePath, fileHandler) {
     const gen = Fill.fileExtentions.get(
        filePath.slice(filePath.lastIndexOf('.'))
     )

    if (!gen) throw new Error('File type not
supportted!')
    this.strategy = new gen(fileHandler)
  }

}
```

Language Javascript

Validators Run all

#1 (click to reveal 1234) Run

#2 (click to reveal 1234) Failed

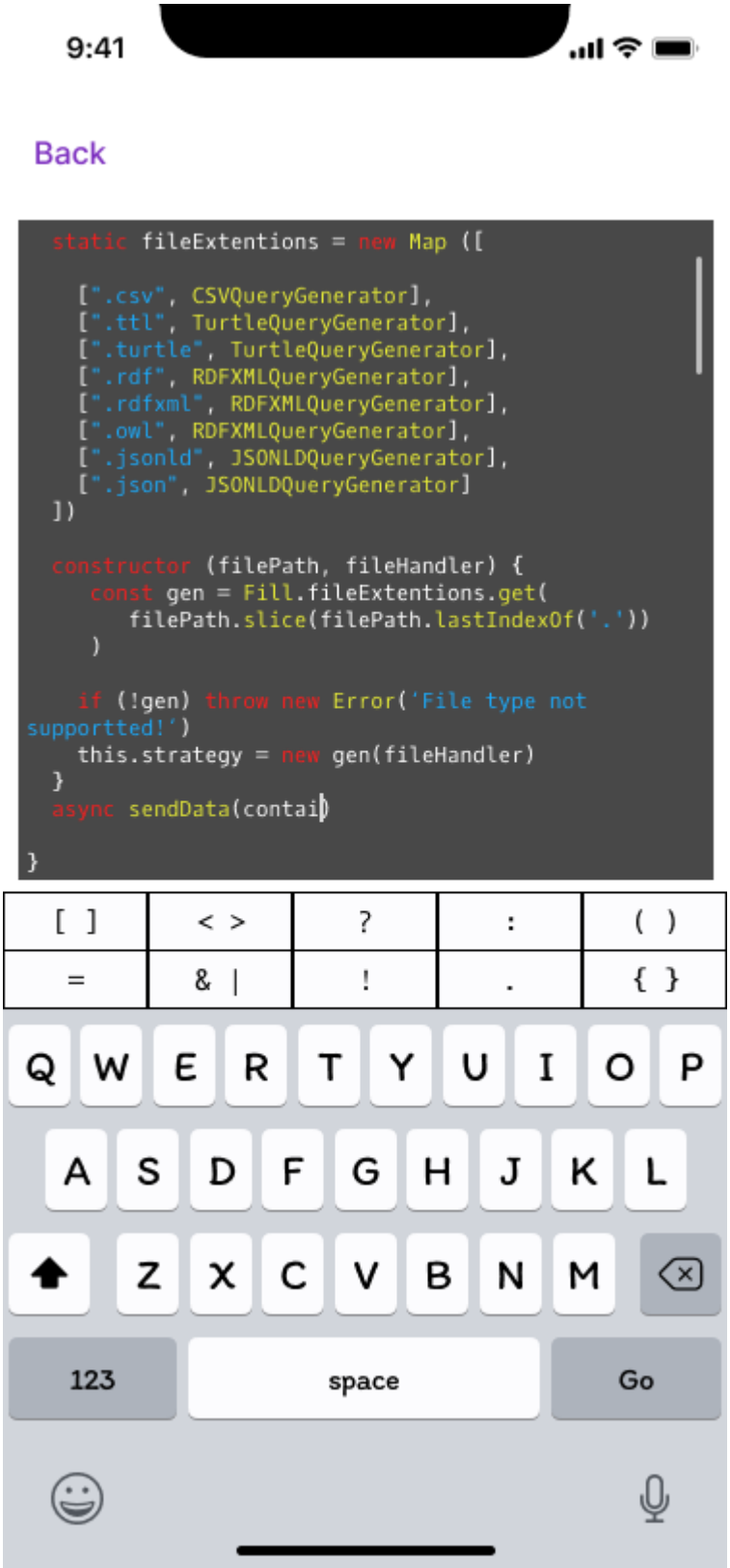
Input: Some Input Here
Output: ereH tupnI emoS Done!

Hints

#1 (click to reveal 1234)

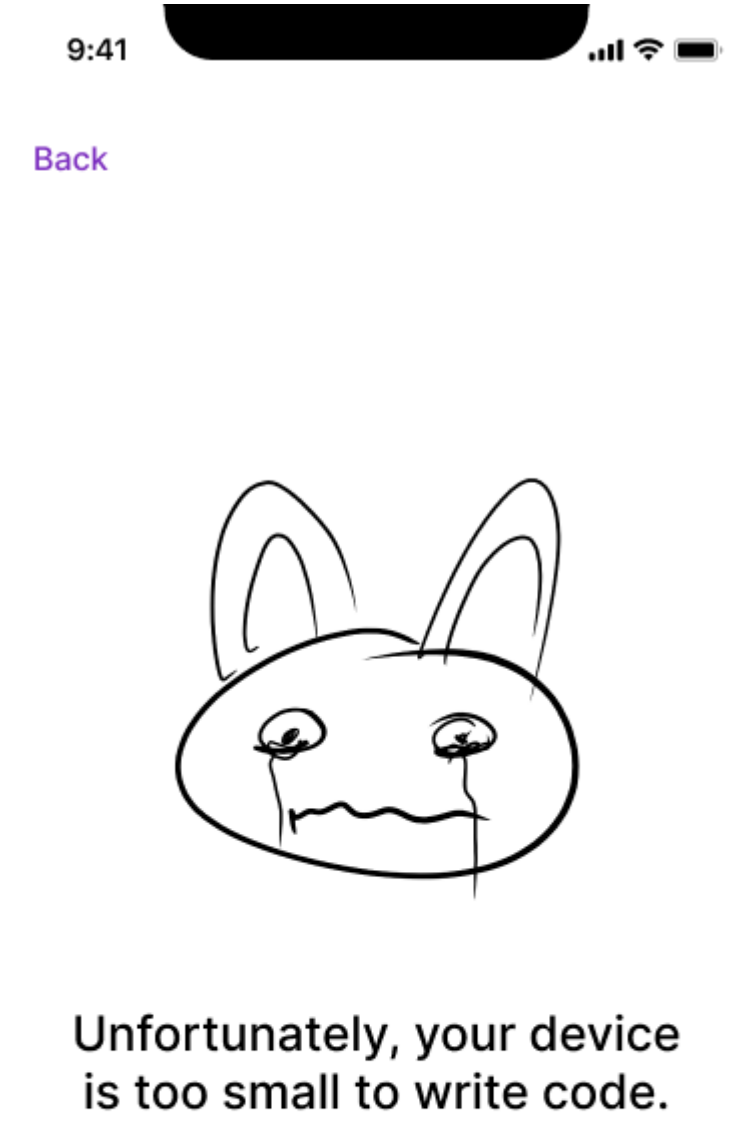
#2 Lorem ipsum dolor sit amet, consectetur a ...

Editeur de code avec focus du clavier



Cependant cette fonctionnalité ne sera pas disponible pour des questions de lisibilité et de confort d'utilisation. Elle sera donc désactivée si l'application mobile juge que votre écran est trop petit, cet inconfort pouvant être considéré comme un désavantage comparé à un ordinateur, les utilisateurs ne pourront pas participer aux campagnes de tests via l'application mobile par soucis d'égalité. Ce problème de visibilité nous contraint aussi à restreindre les opération CRUD de module et de contenu ainsi que d'empêcher l'utilisation des outils d'administration.

Message d'erreur en cas d'écran trop petit



Message d’erreur en cas d’accès à un test

Un mockup UI a été réalisé avec Figma afin de visualiser l'application mobile dans son ensemble. Un aperçu est disponible à cette adresse, vous pouvez naviguer dans le mockup grâce à ce lien.

Flows de fonctionnement

Il y a 5 flows principaux dans l'application mobile:

- L'authentification et la création de compte

- La gestion de compte
- La navigation dans les modules et les contenus
- La gestion d'équipe
- Recherche de contenus et modules

Nous pourrions définirs deux flows supplémentaires, du au fait que nous pourrions ouvrir l'application à partir d'un lien:

- Ouverture d'un lien n'xistant pas (erreur 404)
- Ouverture d'un lien référencant un test qui pour rappel est interdit aux utilisateurs de l'application mobile

API

L'application mobile comuniquera avec notre architecture grâce à une API Rest. Nous aurions pu privilégier GraphQL, un langage de requête orienté client, dans lequel le client formule auprès d'un serveur la forme de sa réponse. Notre application mobile n'étant pas orienté autour de la consommation de données, mais plutôt de l'envoi de données, nous avons préféré utiliser une API Rest, de plus le choix de l'API REST permet de mettre en cache les réponses grâce à l'en-tête HTTP `Headers`.

Comme vous pourrez le voir sur la page Swagger de notre API [api_swagger] que l'API consiste en l'API de la webapp privée de Routes `PATCH`, `PUT` et `DELETE` ces dernières étant des routes modifiant ou supprimant des ressources.

Authentification

Q9: Comment gérer les problématiques de sécurité

Sécurisation du réseau

Pour sécuriser le réseau de notre application nous allons mettre en place une architecture *Zero Trust*, architecture dans laquelle chaque communication entre service est prédéfinie, authentifiée et sécurisée.

premièrement nous allons pouvoir prédéfinir les comminications inter-service un grâce au système intentions de Consul. Ce système permet d'interdire de base tous les inter-connections de nos services dans le cluster, cette approche de *white listing* empêchera à un acteur ayant accès au cluster de crééer des pods potentiellement dangereux et de les connecter à nos autres services.

La seconde des choses à faire dans le cadre de la sécurisation du réseau est de mettre en place un certificat TLS aussi bien pour sécuriser le traffic venant des utilisateurs mais aussi pour sécuriser le traffic à l'intérieur de notre architecture, car si un acteur malveillant parvient à se placer dans notre architecture il pourrait intercepter des données sensibles, ou ce que l'on appelle plus communément une attaque *man in the middle*.

Cependant pour assurrer une meilleure sécurité nous pouvons implémenter un mTLS (multiple TLS), CloudFlare définit le mTLS comme suit dans un article [cloudflare]:

> Mutual TLS, ou mTLS en abrégé, est une méthode d'authentification mutuelle. mTLS garantit que les parties à chaque extrémité d'une connexion réseau sont bien celles qu'elles prétendent être en vérifiant qu'elles possèdent toutes deux la bonne clé privée. Les informations contenues dans leurs certificats TLS respectifs fournissent une vérification supplémentaire. mTLS est souvent utilisé dans un cadre de sécurité Zero Trust pour vérifier les utilisateurs, les appareils et les serveurs au sein d'une organisation. Il peut également contribuer à sécuriser les API.
>
> — Cloudflare, Qu'est-ce que le TLS mutuel (mTLS) ?

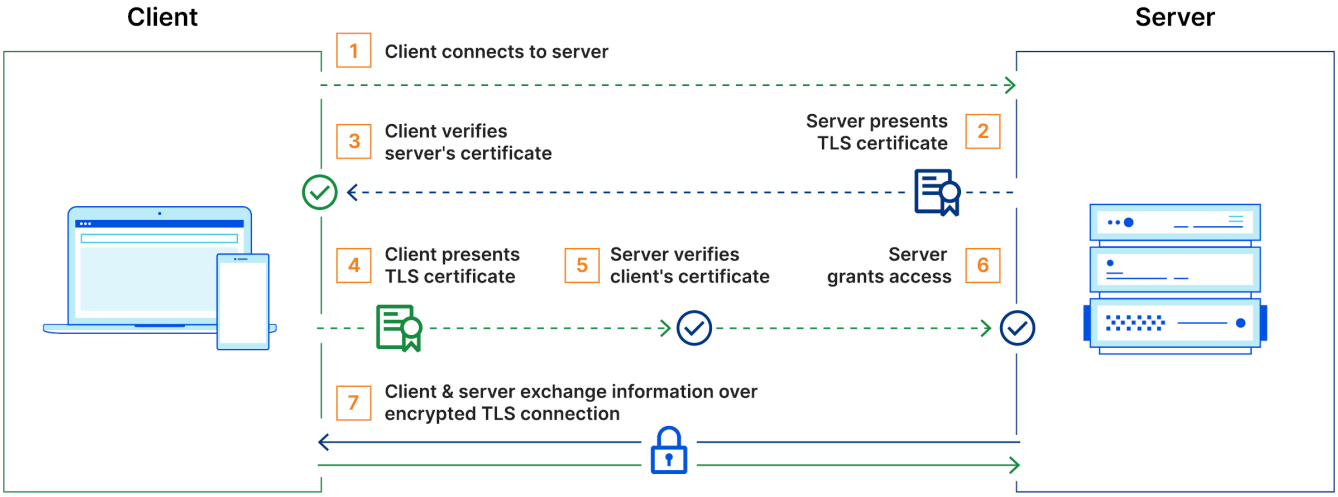
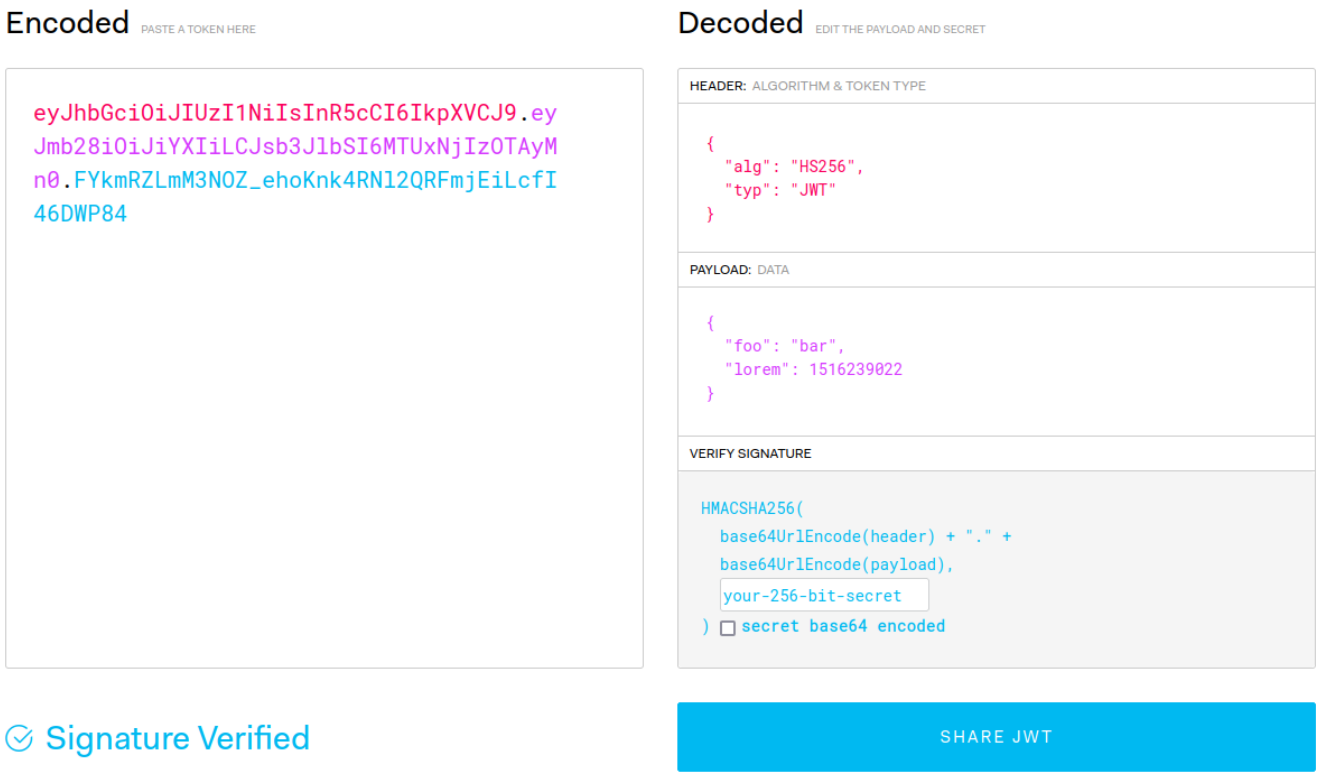


Figure 16. Schématisation de mTLS

Selon ce même article [cloudflare], le mTLS répond à 5 types d'attaque:

- Les attaques *man in the middle*.
- Les usurpations d'identité.
- Les attaques par brute force, qui nécéssiteront encore plus d'essais.
- Le phishing, un attaquant à besoin d'une clé privée et d'un certificat.
- Requêtes venant d'API malveillantes.

En plus de chiffrer nos données envoyées sur le réseau, il faut se charger de vérifier l'intégrité des données si jamais la protection par mTLS est compromise. Pour cela nous allons utiliser des JSON Web Tokens (JWT) qui sont des tokens contenant les données envoyées ainsi qu'une signature permettant de vérifier l'intégrité des données qui consiste en un hachage des données et d'une valeur secrète.



Même si un acteur malveillant venait à modifier les données de nos requêtes (en rouge et rose ci dessus) et à les envoyer, la signature (en bleu) ne serait pas correcte car l'acteur ne connait pas la valeur secrète, nécessaire au calcul de la signature.

Une partie ces protections ont été implémentées dans un POC que vous pourrez retrouver sur GitHub [poc_q9].

Gestion des secrets

Dans notre applications nous aurons de nombreuses variables que nous souhaiterons garder secrètes, comme par exemple les secrets JWT ou encore les identifiants pour nos bases de données. Kubernetes fournit de base des objets Secrets dans son API pour stocker ces valeurs et les sécuriser, pourtant il s'agit de la pire des solutions car les valeurs sont simplement encodés en base64 acessible par n'importe qui ayant un accès au cluster, signifiant que quiconque de mal intentionné s'ayant procuré des accès au cluster aura accès à ces valeurs. Nous allons donc utiliser un gestionnaire de secrets, permettant de chiffrer les secrets et de restreindre l'accès à ces données.

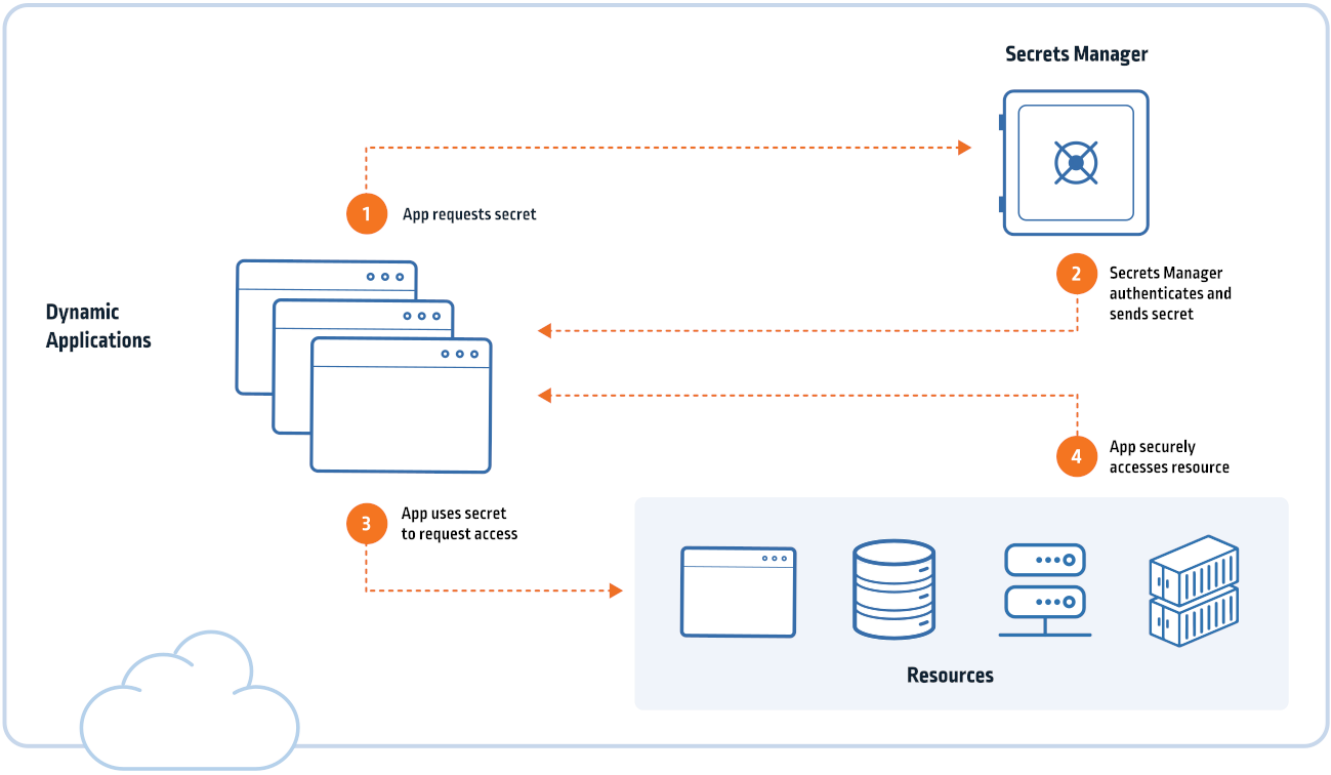


Figure 17. Fonctionnement global d'un gestionnaire de secrets

Lors de recherches sur le sujet les trois noms qui sont le plus ressortis sont Hashicorp Vault, Google Cloud KMS et AWS Secrets Manager. Cependant nous allons choisir Vault car il s'agit de la solution offrant le plus de fonctionnalités intéréssantes possibles:

- Vault est doté d'un moteur PKI permettant alors de génerer des certificats X.509 à la demande, ce qui peut être utilisé dans les cas où nous avons besoins de systèmes de clés publiques/privés. Nous pouvons prendre l'exemple du mTLS où Vault peut servir d'authorité de certificats (Ce qui est totalement possible comme le montre un tutoriel sur leur site [vault_ca]).
- Il permet de générer des secrets dynamiques. C'est à dire que Vault va, à intervalles réguliers, changer certains secrets dont il à la charge tel que les identifiants des bases de données

(changements qu'il va d'ailleur faire appliquer aux SGBD), ce mechanisme sert à contrer les attaques par brute force et à réduire les fenêtres d'attaques car on peut changer les identifiants très fréquemment

- Vault se base sur un système de rôles et d'actions, qui refuse tout accès par défaut. Cet aspect est important quand on sait que les contrôles d'accès défaillants étaient le type de faille présentant le plus de risque dans les systèmes d'information selon le top 10 de l'OWASP en 2021 [owasp10].

Ce sont les raisons pour lesquelles nous utiliserons vault pour sécuriser nos secrets, car les applications auront un accès à des secrets éphémères et régis par des rôles.

Pour ce faire nous utiliseront des *Sidecar* vaults, ce sont des conteneurs placés dans les pods de nos services qui se chargeront de récupérer les secrets auprès de Vault et de les injecter dans les conteneurs du pod.

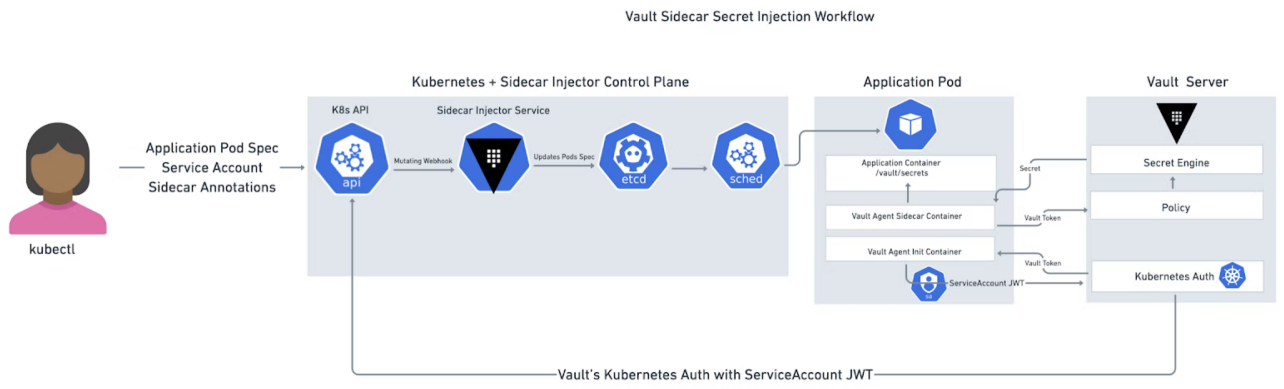


Figure 18. Schématisation de l'injection de secrets

Protection contre les attaques DDoS

Une attaque par déni de service ou attaque DDoS (Distributed Denial of Service) est une cyberattaque qui repose sur un principe simple : Empêcher l'accès à un serveur en le saturant avec des flots de requêtes. Dans un des articles de Cloudflare pévoquant le sujet [cloudflare_dns] on peut retrouver cette analogie

> En généralisant, une attaque DDoS ressemble à un embouteillage inattendu qui bloque une autoroute et empêche le trafic normal d'arriver à destination.
>
> — CloudFlare, Qu'est-ce qu'une attaque DDoS ?

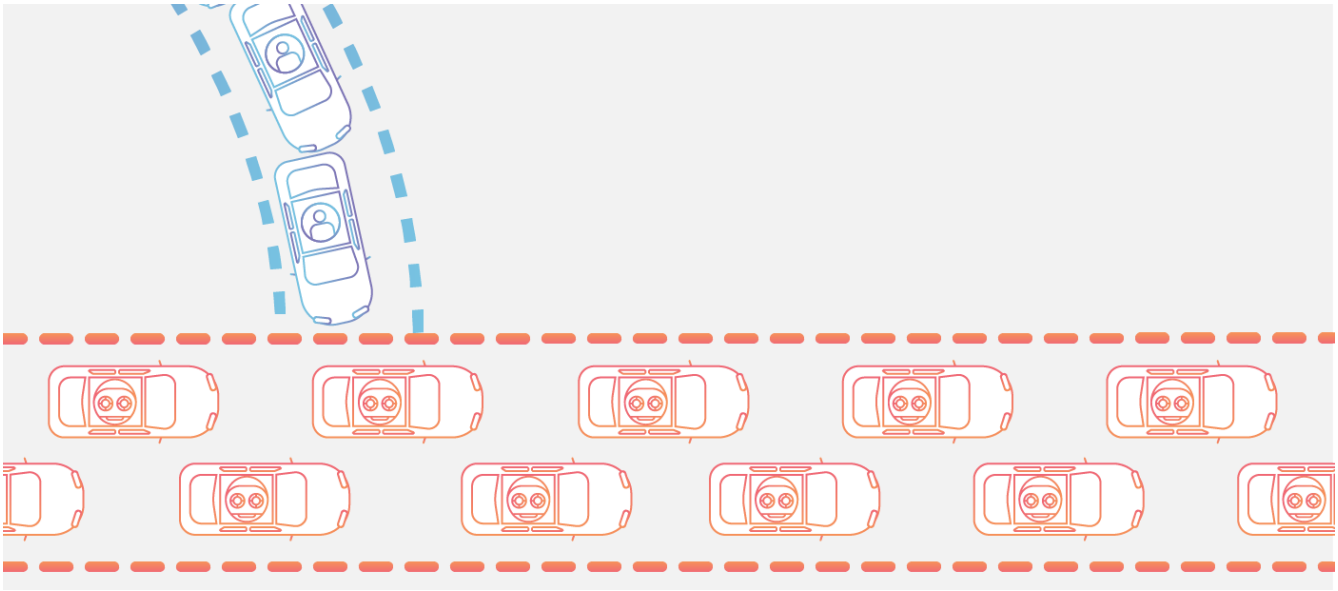


Figure 19. Illustration tirée de l'article de CloudFlare sur le DDoS

Ce type d'attaque à beau sembler primitif, il existe différents types d'attaques DDoS qui exploitent différentes couches du modèle OSI ce qui les rend redoutables et implique la mise en place de quelques dispositifs.

- Les pare feu applicatif (WAF), est une porte d'entrée vers notre application qui va se charger d'évaluer les requêtes entrentes et de les rejeter si elles représentes un danger quelconque comme une attaque DDoS ou une injection.
- La limitation du taux, il est possible de limiter le nombre de requêtes en parallèle, ce qui permet d'empêcher notre application de s'écrouler sous une avalange de requêtes.

Une autre technique pour protéger son application d'attaques DDoS est la technique du trou noir, en cas d'attaque DDoS, tout le traffic peut être redirigé vers une autre destination. Nous n'allons pas garder cette solution car au delà de déplacer le problème autre part, l'attaque reste réussie car tous les clients illégitimes ou non n'ont pas accès à l'application.

Pour appliquer ces techniques nous allons dans un premier cas, utiliser la suite de CloudFlare qui nous permettra d'appliquer un WAF devant l'entrée de notre cluster. Pour ce qui est de la limitation de taux elle peut être configurée dans Consul.

Q10: Comment intégrer les applications UI?

Bibliographie/Webographie/Liens

- [poc_q3_lp] https://gitlab.polytech.umontpellier.fr/yann.pomie/poc3-long-polling
- [poc_q3_ws] https://gitlab.polytech.umontpellier.fr/yann.pomie/poc3-ws
- [poc_q3_http] https://gitlab.polytech.umontpellier.fr/yann.pomie/poc3-http-async
- [rfc6202] https://www.rfc-editor.org/rfc/rfc6202#section-2.2
- [api_swagger] https://woodenmaiden.github.io/rapport_polycode_swagger
- [cloudflare] https://www.cloudflare.com/fr-fr/learning/access-management/what-is-mutual-tls/
- [poc_q9] https://github.com/WoodenMaiden/poc9_polycode
- [vault_ca] https://developer.hashicorp.com/vault/tutorials/secrets-management/pki-engine
- [owasp10] https://owasp.org/Top10/fr/
- [cloudflare_dns] https://www.cloudflare.com/fr-fr/learning/ddos/what-is-a-ddos-attack/